项目代码导出

生成时间: 2026/3/9 14:58:58

文件: src/App.css

```
#root {
  max-width: 1280px;
  margin: 0 auto;
  padding: 2rem;
  text-align: center;
}

.logo {
  height: 6em;
  padding: 1.5em;
  will-change: filter;
  transition: filter 300ms;
}
.logo:hover {
  filter: drop-shadow(0 0 2em #646cffaa);
}
.logo.react:hover {
  filter: drop-shadow(0 0 2em #61dafbaa);
}

@keyframes logo-spin {
  from {
    transform: rotate(0deg);
  }
  to {
    transform: rotate(360deg);
  }
}

@media (prefers-reduced-motion: no-preference) {
  a:nth-of-type(2) .logo {
    animation: logo-spin infinite 20s linear;
  }
}

.card {
  padding: 2em;
}

.read-the-docs {
  color: #888;
}
```

文件: src/components/NavLink.tsx

```
import { NavLink as RouterNavLink, NavLinkProps } from "react-router-dom";
import { forwardRef } from "react";
import { cn } from "@/lib/utils";
```

```
interface NavLinkCompatProps extends Omit<NavLinkProps, "className"> {
  className?: string;
  activeClassName?: string;
  pendingClassName?: string;
}

const NavLink = forwardRef<HTMLAnchorElement, NavLinkCompatProps>(
  ({ className, activeClassName, pendingClassName, to, ...props }, ref) => {
    return (
      <RouterNavLink
        ref={ref}
        to={to}
        className={({ isActive, isPending }) =>
          cn(className, isActive && activeClassName, isPending && pendingClassName)
        }
        {...props}
      />
    );
  },
);

NavLink.displayName = "NavLink";

export { NavLink };
```

文件: src/components/seating/useRoundTableDrag.ts

```
import { useState, useCallback } from 'react';

export type DragSeat = { table: number; seat: number } | null;

export function useRoundTableDrag(
  assignment: string[][],
  setAssignment: React.Dispatch<React.SetStateAction<string[][]>>
) {
  const [dragFrom, setDragFrom] = useState<DragSeat>(null);
  const [dropTarget, setDropTarget] = useState<DragSeat>(null);

  const handleDragStart = useCallback((table: number, seat: number) => {
    setDragFrom({ table, seat });
  }, []);

  const handleDragOver = useCallback((table: number, seat: number) => {
    setDropTarget({ table, seat });
  }, []);

  const handleDrop = useCallback((table: number, seat: number) => {
    if (!dragFrom) return;
    setAssignment(prev => {
      const next = prev.map(t => [...t]);
      // Swap: support cross-table swap
      const fromName = next[dragFrom.table]?.[dragFrom.seat] ?? '';
```

```
      const toName = next[table]?.[seat] ?? '';
      // Ensure arrays are long enough
      while (next[dragFrom.table].length <= dragFrom.seat) next[dragFrom.table].push('');
      while (next[table].length <= seat) next[table].push('');
      next[dragFrom.table][dragFrom.seat] = toName;
      next[table][seat] = fromName;
      return next;
    });
    setDragFrom(null);
    setDropTarget(null);
  }, [dragFrom, setAssignment]);

  const handleDragEnd = useCallback(() => {
    setDragFrom(null);
    setDropTarget(null);
  }, []);

  return { dragFrom, dropTarget, handleDragStart, handleDragOver, handleDrop, handleDragEnd };
}
```

文件: src/components/seating/BanquetHall.tsx

```
import { useState, useRef, useEffect, useMemo, type MouseEvent as ReactMouseEvent } from 'react';
import { Input } from '@/components/ui/input';
import { Button } from '@/components/ui/button';
import { LayoutGrid, Shuffle, QrCode } from 'lucide-react';
import ExportButtons from '@/components/ExportButtons';
import SeatCheckinDialog from '@/components/SeatCheckinDialog';
import { useRoundTableDrag } from './useRoundTableDrag';

interface Props {
  students: { id: string; name: string }[];
}

type BanquetSeatMode = 'tableRoundRobin' | 'tableGrouped' | 'verticalS' | 'horizontalS';
type RefKey = 'screen' | 'podium' | 'window' | 'frontDoor' | 'backDoor';
type RefPositions = Record<RefKey, { x: number; y: number }>;
type RefVisible = Record<RefKey, boolean>;

function buildDefaultRefPositions(roomWidth: number, roomHeight: number): RefPositions {
  const badgeW = 94;
  const centerX = Math.round((roomWidth - badgeW) / 2);
  const rightX = Math.max(24, roomWidth - badgeW - 24);
  const midY = Math.max(20, Math.round((roomHeight - 32) / 2));
  return {
    screen: { x: centerX, y: 22 },
    podium: { x: centerX, y: 74 },
    window: { x: 24, y: midY },
    frontDoor: { x: rightX, y: 120 },
    backDoor: { x: rightX, y: Math.max(180, roomHeight - 56) },
  };
}
```

```
export default function BanquetHall({ students }: Props) {
  const [seatsPerTable, setSeatsPerTable] = useState(10);
  const [tableCount, setTableCount] = useState(() => Math.ceil(students.length / 10) || 3);
  const [groupCount, setGroupCount] = useState(4);
  const [mode, setMode] = useState<BanquetSeatMode>('tableRoundRobin');
  const [assignment, setAssignment] = useState<string[][]>([]);
  const [closedSeats, setClosedSeats] = useState<Set<string>>(new Set());
  const [tableGap, setTableGap] = useState(24);
  const [checkinOpen, setCheckinOpen] = useState(false);
  const [tablePositions, setTablePositions] = useState<{ x: number; y: number }[]>([]);
  const [refVisible, setRefVisible] = useState<RefVisible>({
    screen: true,
    podium: true,
    window: true,
    frontDoor: true,
    backDoor: true,
  });
  const [refLocked, setRefLocked] = useState(false);
  const printRef = useRef<HTMLDivElement>(null);
  const draggingRef = useRef<{ index: number; startX: number; startY: number; origX: number; origY: number } |
null>(null);
  const refDraggingRef = useRef<{ key: RefKey; startX: number; startY: number; origX: number; origY: number } |
null>(null);
  const seatDraggingRef = useRef(false);
  const { dragFrom, dropTarget, handleDragStart, handleDragOver, handleDrop, handleDragEnd } =
useRoundTableDrag(assignment, setAssignment);

  const seatKey = (tableIndex: number, seatIndex: number) => `${tableIndex}-${seatIndex}`;

  const toggleSeatOpen = (tableIndex: number, seatIndex: number) => {
    const key = seatKey(tableIndex, seatIndex);
    setClosedSeats(prev => {
      const next = new Set(prev);
      if (next.has(key)) next.delete(key);
      else next.add(key);
      return next;
    });
  };

  const tableCols = Math.ceil(Math.sqrt(tableCount));
  const tableRows = Math.ceil(tableCount / tableCols);
  const roomWidth = Math.max(980, tableCols * 180 + Math.max(0, tableCols - 1) * tableGap + 260);
  const roomHeight = Math.max(720, tableRows * 180 + Math.max(0, tableRows - 1) * tableGap + 280);
  const tableCellSize = 170;
  const tStageRunwayWidth = 56;
  const hasTStage = refVisible.podium;
  const splitIndex = Math.ceil(tableCols / 2);
  const tStageTopWidth = Math.max(320, Math.min(roomWidth * 0.56, tableCols * 180));
  const tStageTopY = 116;
  const tStageTopHeight = 28;
  const tStageRunwayTop = tStageTopY + tStageTopHeight - 2;
  const tStageRunwayBottom = roomHeight - 64;
  const tStageRunwayHeight = Math.max(120, tStageRunwayBottom - tStageRunwayTop);
  const defaultRefPositions = useMemo(() => buildDefaultRefPositions(roomWidth, roomHeight), [roomWidth,
```

```
roomHeight]);
  const [refPositions, setRefPositions] = useState<RefPositions>(() => buildDefaultRefPositions(980, 720));
  const refBadgeClass = 'absolute h-8 pl-2 pr-2.5 rounded-lg border border-primary/30 bg-primary/10 text-primary
shadow-sm cursor-move select-none inline-flex items-center gap-1.5';
  const refIconClass = 'inline-flex items-center justify-center w-5 h-5 rounded-md border border-primary/30
bg-background/80 text-[11px] leading-none';
  const refTextClass = 'text-[11px] font-medium leading-none tracking-wide';

  const placeName = (tables: string[][], preferred: number, name: string) => {
    const order = [preferred, ...Array.from({ length: tableCount }, (_, i) => i).filter(i => i !== preferred)];
    for (const tableIdx of order) {
      for (let seatIdx = 0; seatIdx < seatsPerTable; seatIdx++) {
        if (closedSeats.has(seatKey(tableIdx, seatIdx))) continue;
        if (!tables[tableIdx][seatIdx]) {
          tables[tableIdx][seatIdx] = name;
          return;
        }
      }
    }
  };

  const splitIntoGroups = (names: string[], count: number) => {
    const groups: string[][] = Array.from({ length: count }, () => []);
    names.forEach((n, i) => groups[i % count].push(n));
    return groups;
  };

  const getTableOrder = (seatMode: BanquetSeatMode) => {
    const cols = Math.ceil(Math.sqrt(tableCount));
    const rows = Math.ceil(tableCount / cols);
    const order: number[] = [];

    if (seatMode === 'verticalS') {
      for (let c = 0; c < cols; c++) {
        for (let ri = 0; ri < rows; ri++) {
          const r = c % 2 === 0 ? ri : rows - 1 - ri;
          const idx = r * cols + c;
          if (idx < tableCount) order.push(idx);
        }
      }
      return order;
    }

    if (seatMode === 'horizontalS') {
      for (let r = 0; r < rows; r++) {
        for (let ci = 0; ci < cols; ci++) {
          const c = r % 2 === 0 ? ci : cols - 1 - ci;
          const idx = r * cols + c;
          if (idx < tableCount) order.push(idx);
        }
      }
      return order;
    }
```

```
  return Array.from({ length: tableCount }, (_, i) => i);
};

const autoSeat = (shuffle = false) => {
  const names = shuffle
    ? [...students.map(s => s.name)].sort(() => Math.random() - 0.5)
    : students.map(s => s.name);
  const tables: string[][] = Array.from({ length: tableCount }, () => Array.from({ length: seatsPerTable }, () => ''));

  if (mode === 'tableGrouped') {
    const groups = splitIntoGroups(names, Math.max(1, groupCount));
    groups.forEach((group, gi) => {
      group.forEach(n => placeName(tables, gi % tableCount, n));
    });
    setAssignment(tables);
    return;
  }

  const order = getTableOrder(mode);
  names.forEach((n, i) => placeName(tables, order[i % order.length], n));
  setAssignment(tables);
};

useEffect(() => {
  setTablePositions(Array(tableCount).fill({ x: 0, y: 0 }));
}, [tableCount]);

useEffect(() => {
  const matchedTableCount = Math.max(1, Math.ceil(students.length / Math.max(1, seatsPerTable)));
  setTableCount(matchedTableCount);
}, [students.length, seatsPerTable]);

useEffect(() => {
  setRefPositions(defaultRefPositions);
}, [defaultRefPositions]);

useEffect(() => {
  setClosedSeats(prev => {
    const next = new Set<string>();
    prev.forEach(key => {
      const [tableStr, seatStr] = key.split('-');
      const t = Number(tableStr);
      const s = Number(seatStr);
      if (t < tableCount && s < seatsPerTable) next.add(key);
    });
    return next;
  });
}, [tableCount, seatsPerTable]);

useEffect(() => {
  const handleMouseMove = (e: MouseEvent) => {
    if (draggingRef.current) {
      const dx = e.clientX - draggingRef.current.startX;
      const dy = e.clientY - draggingRef.current.startY;
```

```
      setTablePositions(pos => pos.map((p, i) =>
        i === draggingRef.current!.index
          ? { x: draggingRef.current!.origX + dx, y: draggingRef.current!.origY + dy }
          : p
      ));
    }

    if (refDraggingRef.current) {
      const dx = e.clientX - refDraggingRef.current.startX;
      const dy = e.clientY - refDraggingRef.current.startY;
      const key = refDraggingRef.current.key;
      setRefPositions(prev => ({
        ...prev,
        [key]: {
          x: refDraggingRef.current!.origX + dx,
          y: refDraggingRef.current!.origY + dy,
        },
      }));
    }
  };
  const handleMouseUp = () => {
    draggingRef.current = null;
    refDraggingRef.current = null;
  };
  window.addEventListener('mousemove', handleMouseMove);
  window.addEventListener('mouseup', handleMouseUp);
  return () => {
    window.removeEventListener('mousemove', handleMouseMove);
    window.removeEventListener('mouseup', handleMouseUp);
  };
}, []);

const startTableDrag = (e: React.MouseEvent, index: number) => {
  e.stopPropagation();
  if (seatDraggingRef.current) return;
  draggingRef.current = {
    index,
    startX: e.clientX,
    startY: e.clientY,
    origX: tablePositions[index]?.x || 0,
    origY: tablePositions[index]?.y || 0,
  };
};

const toggleRefVisible = (key: RefKey) => {
  setRefVisible(prev => ({ ...prev, [key]: !prev[key] }));
};

const startRefDrag = (e: ReactMouseEvent, key: RefKey) => {
  if (refLocked) return;
  e.preventDefault();
  e.stopPropagation();
  refDraggingRef.current = {
    key,
```

```
    startX: e.clientX,
    startY: e.clientY,
    origX: refPositions[key].x,
    origY: refPositions[key].y,
  };
};

const renderBanquetTable = (tableIndex: number, people: string[]) => {
  const radius = 60;
  const seatRadius = 16;
  const cx = 85;
  const cy = 85;
  const totalSlots = seatsPerTable;
  const pos = tablePositions[tableIndex] || { x: 0, y: 0 };

  return (
    <div
      key={tableIndex}
      className="flex flex-col items-center cursor-move"
      style={{ transform: `translate(${pos.x}px,${pos.y}px)` }}
      onMouseDown={e => startTableDrag(e, tableIndex)}
    >
      <svg width={170} height={170} viewBox="0 0 170 170" className="font-sans" style={{ fontFamily:
'var(--font-family)' }}>
        <circle cx={cx} cy={cy} r={42} className="fill-primary/5 stroke-primary/20" strokeWidth={1} strokeDasharray="4 2"
/>
        <circle cx={cx} cy={cy} r={36} className="fill-primary/10 stroke-primary/30" strokeWidth={2} />
        <text x={cx} y={cy - 4} textAnchor="middle" dominantBaseline="middle" className="fill-primary text-sm
font-medium">
          {tableIndex + 1}桌
        </text>
        <text x={cx} y={cy + 8} textAnchor="middle" dominantBaseline="middle" className="fill-primary/60 text-xs">
          {people.length}人
        </text>
        {Array.from({ length: totalSlots }).map((_, i) => {
          const angle = (2 * Math.PI * i) / totalSlots - Math.PI / 2;
          const sx = cx + radius * Math.cos(angle);
          const sy = cy + radius * Math.sin(angle);
          const name = people[i] || '';
          const isClosed = closedSeats.has(seatKey(tableIndex, i));
          const isDragging = dragFrom?.table === tableIndex && dragFrom?.seat === i;
          const isOver = dropTarget?.table === tableIndex && dropTarget?.seat === i;
          return (
            <g
              key={i}
              style={{ cursor: name && !isClosed ? 'grab' : 'pointer' }}
              onMouseDown={name && !isClosed ? (e) => {
                e.preventDefault();
                e.stopPropagation();
                seatDraggingRef.current = true;
                handleDragStart(tableIndex, i);
              } : undefined}
              onMouseEnter={() => { if (dragFrom && !isClosed) handleDragOver(tableIndex, i); }}
              onMouseUp={() => {
```

```
            if (dragFrom && !isClosed) handleDrop(tableIndex, i);
            seatDraggingRef.current = false;
          }}
          onClick={() => { if (!name) toggleSeatOpen(tableIndex, i); }}
        >
          <circle
            cx={sx}
            cy={sy}
            r={seatRadius}
            className={
              isClosed ? 'fill-muted stroke-destructive/60' :
              isDragging ? 'fill-primary/20 stroke-primary' :
              isOver ? 'fill-accent stroke-primary' :
              name ? 'fill-card stroke-border hover:stroke-primary/50' : 'fill-muted/30 stroke-border/30'
            }
            strokeWidth={isOver ? 2.5 : 1.5}
            style={{ transition: 'all 0.15s' }}
          />
          {isClosed && (
            <text x={sx} y={sy + 1} textAnchor="middle" dominantBaseline="middle" className="fill-destructive text-xs
pointer-events-none">
              关
            </text>
          )}
          {name && !isDragging && (
            <text x={sx} y={sy + 1} textAnchor="middle" dominantBaseline="middle" className="fill-foreground text-xs
pointer-events-none">
              {name.length > 3 ? name.slice(0, 3) : name}
            </text>
          )}
        </g>
      );
    })}
  </svg>
</div>
);
};

return (
<div
  onMouseUp={() => {
    handleDragEnd();
    seatDraggingRef.current = false;
  }}
  onMouseLeave={() => {
    handleDragEnd();
    seatDraggingRef.current = false;
  }}
>
  <div className="flex flex-wrap items-center gap-3 mb-5">
    <label className="flex items-center gap-2 text-sm text-muted-foreground">
      每桌人数
      <Input type="number" min={6} max={16} value={seatsPerTable}
        onChange={e => setSeatsPerTable(Math.max(6, Math.min(16, Number(e.target.value))))} className="w-16 h-8
```

```
text-center" />
      </label>
      <label className="flex items-center gap-2 text-sm text-muted-foreground">
        自动桌数
        <span className="inline-flex items-center justify-center min-w-10 h-8 px-2 rounded-md border border-border
bg-muted/40 text-foreground font-medium">
          {tableCount}
        </span>
      </label>
      <label className="flex items-center gap-2 text-sm text-muted-foreground">
        模式
        <select
          value={mode}
          onChange={e => setMode(e.target.value as BanquetSeatMode)}
          className="h-8 px-2 rounded-md border border-input bg-background text-foreground text-sm"
        >
          <option value="tableRoundRobin">每桌轮转</option>
          <option value="tableGrouped">每组一桌</option>
          <option value="verticalS">竖S桌序</option>
          <option value="horizontalS">横S桌序</option>
        </select>
      </label>
      {mode === 'tableGrouped' && (
        <label className="flex items-center gap-2 text-sm text-muted-foreground">
          组数
          <Input type="number" min={2} max={30} value={groupCount}
            onChange={e => setGroupCount(Math.max(2, Math.min(30, Number(e.target.value))))} className="w-16 h-8
text-center" />
        </label>
      )}
      <label className="flex items-center gap-2 text-sm text-muted-foreground">
        桌子间距
        <Input type="number" min={0} max={100} value={tableGap}
          onChange={e => setTableGap(Math.max(0, Math.min(100, Number(e.target.value))))} className="w-16 h-8
text-center" />
      </label>
      <Button variant="outline" onClick={() => setRefPositions(defaultRefPositions)}>
        重置参照物
      </Button>
      <div className="flex items-center gap-2 text-xs text-muted-foreground">
        <label className="flex items-center gap-1 cursor-pointer">
          <input type="checkbox" checked={refVisible.screen} onChange={() => toggleRefVisible('screen')}
className="accent-primary" /> 幕布
        </label>
        <label className="flex items-center gap-1 cursor-pointer">
          <input type="checkbox" checked={refVisible.podium} onChange={() => toggleRefVisible('podium')}
className="accent-primary" /> T型舞台
        </label>
        <label className="flex items-center gap-1 cursor-pointer">
          <input type="checkbox" checked={refVisible.window} onChange={() => toggleRefVisible('window')}
className="accent-primary" /> 窗
        </label>
        <label className="flex items-center gap-1 cursor-pointer">
          <input type="checkbox" checked={refVisible.frontDoor} onChange={() => toggleRefVisible('frontDoor')}
```

```
className="accent-primary" />前门
        </label>
        <label className="flex items-center gap-1 cursor-pointer">
          <input type="checkbox" checked={refVisible.backDoor} onChange={() => toggleRefVisible('backDoor')}
className="accent-primary" />后门
        </label>
        <label className="flex items-center gap-1 cursor-pointer">
          <input type="checkbox" checked={refLocked} onChange={e => setRefLocked(e.target.checked)}
className="accent-primary" />锁定参照物
        </label>
      </div>
      <span className="text-xs text-muted-foreground">
        共可容纳 {seatsPerTable * tableCount} 人 | 当前 {students.length} 人
      </span>
      {assignment.length > 0 && <ExportButtons targetRef={printRef} filename="宴会厅座位" />}
      {assignment.length > 0 && (
        <Button variant="outline" onClick={() => setCheckinOpen(true)} className="gap-2">
          <QrCode className="w-4 h-4" />签到
        </Button>
      )}
      <div className="flex gap-2 ml-auto">
        <Button variant="outline" onClick={() => autoSeat(true)} className="gap-2">
          <Shuffle className="w-4 h-4" />随机排座
        </Button>
        <Button onClick={() => autoSeat(false)} className="gap-2">
          <LayoutGrid className="w-4 h-4" />自动排座
        </Button>
      </div>
    </div>

    <div ref={printRef}>
      <div className="text-center mb-4">
        <div className="inline-block bg-primary/10 text-primary px-6 py-2 rounded-lg text-sm font-medium border
border-primary/20">
          宴会厅
        </div>
      </div>

      {assignment.length > 0 ? (
        <div className="flex justify-center overflow-auto">
          <div className="relative rounded-xl border border-border bg-card/40" style={{ width: roomWidth, height:
roomHeight }}>
            {refVisible.screen && (
              <div className={refBadgeClass} style={{ left: refPositions.screen.x, top: refPositions.screen.y }}
onMouseDown={e => startRefDrag(e, 'screen')}>
                <span className={refIconClass}></span>
                <span className={refTextClass}>幕布</span>
              </div>
            )}
            {hasTStage && (
              <>
                <div
                  className="absolute rounded-xl border border-primary/35 bg-primary/12 shadow-sm
pointer-events-none"
```

```
            style={{
              left: `calc(50% - ${tStageTopWidth / 2}px)`,
              top: tStageTopY,
              width: tStageTopWidth,
              height: tStageTopHeight,
            }}
          />
          <div
            className="absolute rounded-xl border border-primary/35 bg-primary/12 shadow-sm
pointer-events-none"
            style={{
              left: `calc(50% - ${tStageRunwayWidth / 2}px)`,
              top: tStageRunwayTop,
              width: tStageRunwayWidth,
              height: tStageRunwayHeight,
            }}
          />
          <div
            className="absolute text-[11px] font-medium text-primary/80 select-none pointer-events-none"
            style={{ left: '50%', top: tStageTopY + 6, transform: 'translateX(-50%)' }}
          >
            T型舞台
          </div>
        </>
      )}
      {refVisible.window && (
        <div className={refBadgeClass} style={{ left: refPositions.window.x, top: refPositions.window.y }}
onMouseDown={e => startRefDrag(e, 'window')}>
          <span className={refIconClass}></span>
          <span className={refTextClass}>窗</span>
        </div>
      )}
      {refVisible.frontDoor && (
        <div className={refBadgeClass} style={{ left: refPositions.frontDoor.x, top: refPositions.frontDoor.y }}
onMouseDown={e => startRefDrag(e, 'frontDoor')}>
          <span className={refIconClass}></span>
          <span className={refTextClass}>前门</span>
        </div>
      )}
      {refVisible.backDoor && (
        <div className={refBadgeClass} style={{ left: refPositions.backDoor.x, top: refPositions.backDoor.y }}
onMouseDown={e => startRefDrag(e, 'backDoor')}>
          <span className={refIconClass}></span>
          <span className={refTextClass}>后门</span>
        </div>
      )}

      <div className="absolute inset-0 flex items-center justify-center pointer-events-none">
        <div
          className="inline-grid pointer-events-auto"
          style={{
            gridTemplateColumns: hasTStage && tableCols > 1
              ? `repeat(${splitIndex}, ${tableCellSize}px) ${tStageRunwayWidth}px repeat(${tableCols - splitIndex},
${tableCellSize}px)`
```

```
              : `repeat(${tableCols}, ${tableCellSize}px)`,
            columnGap: `${tableGap}px`,
            rowGap: `${tableGap}px`,
          }}
        >
          {assignment.map((people, i) => {
            const col = i % tableCols;
            const row = Math.floor(i / tableCols);
            const visualCol = hasTStage && tableCols > 1 && col >= splitIndex ? col + 2 : col + 1;
            return (
              <div key={`banquet-table-cell-${i}`} style={{ gridColumn: visualCol, gridRow: row + 1 }}>
                {renderBanquetTable(i, people)}
              </div>
            );
          })}
        </div>
      </div>
    </div>
  ) : (
    <div className="text-center py-20 text-muted-foreground">
      <p className="text-lg mb-2">点击「自动排座」开始安排</p>
      <p className="text-sm">宴会厅圆桌，{seatsPerTable} 人一桌，根据人数自动分配</p>
    </div>
  )}
</div>

{assignment.length > 0 && (
  <p className="text-center text-xs text-muted-foreground mt-4">
    拖拽姓名可交换座位；点击空座位可关闭/开放使用；幕布/T型舞台/窗/前后门支持显隐与拖拽
  </p>
)}
<SeatCheckinDialog
  open={checkinOpen}
  onOpenChange={setCheckinOpen}
  seatData={assignment}
  studentNames={students.map(s => s.name)}
  sceneType="banquet"
  sceneConfig={{ seatsPerTable, tableCount, tableCols: Math.ceil(Math.sqrt(tableCount)) }}
/>
</div>
);
}
```

文件: src/components/seating/SmartClassroom.tsx

```
import { useState, useRef, useEffect, useMemo } from 'react';
import { Input } from '@/components/ui/input';
import { Button } from '@/components/ui/button';
import { LayoutGrid, Shuffle, QrCode } from 'lucide-react';
import ExportButtons from '@/components/ExportButtons';
import SeatCheckinDialog from '@/components/SeatCheckinDialog';
import { useRoundTableDrag } from './useRoundTableDrag';
```

```
interface Props {
  students: { id: string; name: string }[];
}

type SmartSeatMode = 'tableRoundRobin' | 'tableGrouped' | 'verticalS' | 'horizontalS';
type RefKey = 'screen' | 'podium' | 'frontDoor' | 'backDoor' | 'window';
type RefPositions = Record<RefKey, { x: number; y: number }>;
type RefVisible = Record<RefKey, boolean>;

function getDefaultRefPositions(roomWidth: number, roomHeight: number): RefPositions {
  const badgeW = 94;
  const centeredX = Math.round((roomWidth - badgeW) / 2);
  const rightX = Math.max(24, roomWidth - badgeW - 24);
  const centerY = Math.max(20, Math.round((roomHeight - 32) / 2));
  return {
    screen: { x: centeredX, y: 22 },
    podium: { x: centeredX, y: 74 },
    frontDoor: { x: rightX, y: 120 },
    backDoor: { x: rightX, y: Math.max(160, roomHeight - 56) },
    window: { x: 24, y: centerY },
  };
}

export default function SmartClassroom({ students }: Props) {
  const [seatsPerTable, setSeatsPerTable] = useState(6);
  const [tableCount, setTableCount] = useState(() => Math.ceil(students.length / 6) || 4);
  const [groupCount, setGroupCount] = useState(4);
  const [mode, setMode] = useState<SmartSeatMode>('tableRoundRobin');
  const [assignment, setAssignment] = useState<string[][]>([]);
  const [closedSeats, setClosedSeats] = useState<Set<string>>(new Set());
  const [tableGap, setTableGap] = useState(20);
  const [tablePositions, setTablePositions] = useState<{ x: number; y: number }[]>([]);
  const [refPositions, setRefPositions] = useState<RefPositions>(() => getDefaultRefPositions(920, 640));
  const [refVisible, setRefVisible] = useState<RefVisible>({
    screen: true,
    podium: true,
    frontDoor: true,
    backDoor: true,
    window: true,
  });
  const [refLocked, setRefLocked] = useState(false);
  const [checkinOpen, setCheckinOpen] = useState(false);

  const printRef = useRef<HTMLDivElement>(null);
  const draggingRef = useRef<{ index: number; startX: number; startY: number; origX: number; origY: number } |
null>(null);
  const refDraggingRef = useRef<{ key: RefKey; startX: number; startY: number; origX: number; origY: number } |
null>(null);
  const seatDraggingRef = useRef(false);

  const { dragFrom, dropTarget, handleDragStart, handleDragOver, handleDrop, handleDragEnd } =
useRoundTableDrag(assignment, setAssignment);
```

```
const seatKey = (tableIndex: number, seatIndex: number) => `${tableIndex}-${seatIndex}`;

const toggleSeatOpen = (tableIndex: number, seatIndex: number) => {
  const key = seatKey(tableIndex, seatIndex);
  setClosedSeats(prev => {
    const next = new Set(prev);
    if (next.has(key)) next.delete(key);
    else next.add(key);
    return next;
  });
};

const toggleRefVisible = (key: RefKey) => {
  setRefVisible(prev => ({ ...prev, [key]: !prev[key] }));
};

const placeName = (tables: string[][], preferred: number, name: string) => {
  const order = [preferred, ...Array.from({ length: tableCount }, (_, i) => i).filter(i => i !== preferred)];
  for (const tableIdx of order) {
    for (let seatIdx = 0; seatIdx < seatsPerTable; seatIdx++) {
      if (closedSeats.has(seatKey(tableIdx, seatIdx))) continue;
      if (!tables[tableIdx][seatIdx]) {
        tables[tableIdx][seatIdx] = name;
        return;
      }
    }
  }
};

const splitIntoGroups = (names: string[], count: number) => {
  const groups: string[][] = Array.from({ length: count }, () => []);
  names.forEach((n, i) => groups[i % count].push(n));
  return groups;
};

const getTableOrder = (seatMode: SmartSeatMode) => {
  const cols = Math.ceil(Math.sqrt(tableCount));
  const rows = Math.ceil(tableCount / cols);
  const order: number[] = [];

  if (seatMode === 'verticalS') {
    for (let c = 0; c < cols; c++) {
      for (let ri = 0; ri < rows; ri++) {
        const r = c % 2 === 0 ? ri : rows - 1 - ri;
        const idx = r * cols + c;
        if (idx < tableCount) order.push(idx);
      }
    }
    return order;
  }

  if (seatMode === 'horizontalS') {
    for (let r = 0; r < rows; r++) {
      for (let ci = 0; ci < cols; ci++) {
```

```
        const c = r % 2 === 0 ? ci : cols - 1 - ci;
        const idx = r * cols + c;
        if (idx < tableCount) order.push(idx);
      }
    }
    return order;
  }

  return Array.from({ length: tableCount }, (_, i) => i);
};

const autoSeat = (shuffle = false) => {
  const names = shuffle
    ? [...students.map(s => s.name)].sort(() => Math.random() - 0.5)
    : students.map(s => s.name);
  const tables: string[][] = Array.from({ length: tableCount }, () => Array.from({ length: seatsPerTable }, () => ''));

  if (mode === 'tableGrouped') {
    const groups = splitIntoGroups(names, Math.max(1, groupCount));
    groups.forEach((group, gi) => {
      group.forEach(n => placeName(tables, gi % tableCount, n));
    });
    setAssignment(tables);
    return;
  }

  const order = getTableOrder(mode);
  names.forEach((n, i) => placeName(tables, order[i % order.length], n));
  setAssignment(tables);
};

const tableCols = Math.ceil(Math.sqrt(tableCount));
const tableRows = Math.ceil(tableCount / tableCols);
const roomWidth = Math.max(920, tableCols * 160 + Math.max(0, tableCols - 1) * tableGap + 220);
const roomHeight = Math.max(640, tableRows * 160 + Math.max(0, tableRows - 1) * tableGap + 240);
const defaultRefPositions = useMemo(() => getDefaultRefPositions(roomWidth, roomHeight), [roomWidth,
roomHeight]);
const refBadgeClass = 'absolute h-8 pl-2 pr-2.5 rounded-lg border border-primary/30 bg-primary/10 text-primary
shadow-sm cursor-move select-none inline-flex items-center gap-1.5';
const refIconClass = 'inline-flex items-center justify-center w-5 h-5 rounded-md border border-primary/30
bg-background/80 text-[11px] leading-none';
const refTextClass = 'text-[11px] font-medium leading-none tracking-wide';

useEffect(() => {
  const matchedTableCount = Math.max(1, Math.ceil(students.length / Math.max(1, seatsPerTable)));
  setTableCount(matchedTableCount);
}, [students.length, seatsPerTable]);

useEffect(() => {
  setRefPositions(defaultRefPositions);
}, [defaultRefPositions]);

useEffect(() => {
  setTablePositions(Array(tableCount).fill({ x: 0, y: 0 }));
```

```
}, [tableCount]);

useEffect(() => {
  setClosedSeats(prev => {
    const next = new Set<string>();
    prev.forEach(key => {
      const [tableStr, seatStr] = key.split('-');
      const t = Number(tableStr);
      const s = Number(seatStr);
      if (t < tableCount && s < seatsPerTable) next.add(key);
    });
    return next;
  });
}, [tableCount, seatsPerTable]);

useEffect(() => {
  const handleMouseMove = (e: MouseEvent) => {
    if (draggingRef.current) {
      const dx = e.clientX - draggingRef.current.startX;
      const dy = e.clientY - draggingRef.current.startY;
      setTablePositions(pos => pos.map((p, i) =>
        i === draggingRef.current!.index
          ? { x: draggingRef.current!.origX + dx, y: draggingRef.current!.origY + dy }
          : p
      ));
    }

    if (refDraggingRef.current) {
      const dx = e.clientX - refDraggingRef.current.startX;
      const dy = e.clientY - refDraggingRef.current.startY;
      const key = refDraggingRef.current.key;
      setRefPositions(prev => ({
        ...prev,
        [key]: {
          x: refDraggingRef.current!.origX + dx,
          y: refDraggingRef.current!.origY + dy,
        },
      }));
    }
  };

  const handleMouseUp = () => {
    draggingRef.current = null;
    refDraggingRef.current = null;
  };

  window.addEventListener('mousemove', handleMouseMove);
  window.addEventListener('mouseup', handleMouseUp);
  return () => {
    window.removeEventListener('mousemove', handleMouseMove);
    window.removeEventListener('mouseup', handleMouseUp);
  };
}, []);
```

```
  const startTableDrag = (e: React.MouseEvent, index: number) => {
    e.stopPropagation();
    if (seatDraggingRef.current) return;
    draggingRef.current = {
      index,
      startX: e.clientX,
      startY: e.clientY,
      origX: tablePositions[index]?.x || 0,
      origY: tablePositions[index]?.y || 0,
    };
  };

  const startRefDrag = (e: React.MouseEvent, key: RefKey) => {
    if (refLocked) return;
    e.preventDefault();
    e.stopPropagation();
    refDraggingRef.current = {
      key,
      startX: e.clientX,
      startY: e.clientY,
      origX: refPositions[key].x,
      origY: refPositions[key].y,
    };
  };

  const renderRoundTable = (tableIndex: number, people: string[]) => {
    const radius = 52;
    const seatRadius = 16;
    const cx = 80;
    const cy = 80;
    const totalSlots = seatsPerTable;
    const pos = tablePositions[tableIndex] || { x: 0, y: 0 };

    return (
      <div
        key={tableIndex}
        className="flex flex-col items-center gap-1 cursor-move"
        style={{ transform: `translate(${pos.x}px,${pos.y}px)` }}
        onMouseDown={e => startTableDrag(e, tableIndex)}
      >
        <svg width={160} height={160} viewBox="0 0 160 160" className="font-sans" style={{ fontFamily:
'var(--font-family)' }}>
          <circle cx={cx} cy={cy} r={36} className="fill-primary/10 stroke-primary/30" strokeWidth={2} />
          <text x={cx} y={cy + 1} textAnchor="middle" dominantBaseline="middle" className="fill-primary text-xs
font-medium">
            {tableIndex + 1}桌
          </text>
          {Array.from({ length: totalSlots }).map((_, i) => {
            const angle = (2 * Math.PI * i) / totalSlots - Math.PI / 2;
            const sx = cx + radius * Math.cos(angle);
            const sy = cy + radius * Math.sin(angle);
            const name = people[i] || '';
            const isClosed = closedSeats.has(seatKey(tableIndex, i));
            const isDragging = dragFrom?.table === tableIndex && dragFrom?.seat === i;
```

```
          const isOver = dropTarget?.table === tableIndex && dropTarget?.seat === i;
          return (
            <g
              key={i}
              style={{ cursor: name && !isClosed ? 'grab' : 'pointer' }}
              onMouseDown={name && !isClosed ? (e) => {
                e.preventDefault();
                e.stopPropagation();
                seatDraggingRef.current = true;
                handleDragStart(tableIndex, i);
              } : undefined}
              onMouseEnter={() => { if (dragFrom && !isClosed) handleDragOver(tableIndex, i); }}
              onMouseUp={() => {
                if (dragFrom && !isClosed) handleDrop(tableIndex, i);
                seatDraggingRef.current = false;
              }}
              onClick={() => { if (!name) toggleSeatOpen(tableIndex, i); }}
            >
              <circle
                cx={sx} cy={sy} r={seatRadius}
                className={
                  isClosed ? 'fill-muted stroke-destructive/60' :
                  isDragging ? 'fill-primary/20 stroke-primary' :
                  isOver ? 'fill-accent stroke-primary' :
                  name ? 'fill-card stroke-border hover:stroke-primary/50' : 'fill-muted/50 stroke-border/50'
                }
                strokeWidth={isOver ? 2.5 : 1.5}
                style={{ transition: 'all 0.15s' }}
              />
              {isClosed && (
                <text x={sx} y={sy + 1} textAnchor="middle" dominantBaseline="middle" className="fill-destructive text-xs
pointer-events-none">
                  关
                </text>
              )}
              {name && !isDragging && (
                <text x={sx} y={sy + 1} textAnchor="middle" dominantBaseline="middle" className="fill-foreground text-xs
pointer-events-none">
                  {name.length > 3 ? name.slice(0, 3) : name}
                </text>
              )}
            </g>
          );
        })}
      </svg>
    </div>
  );
};

return (
  <div
    onMouseUp={() => {
      handleDragEnd();
      seatDraggingRef.current = false;
```

```
      refDraggingRef.current = null;
    }}
    onMouseLeave={() => {
      handleDragEnd();
      seatDraggingRef.current = false;
      refDraggingRef.current = null;
    }}
  >
    <div className="flex flex-wrap items-center gap-3 mb-5">
      <label className="flex items-center gap-2 text-sm text-muted-foreground">
        每桌人数
        <Input type="number" min={3} max={12} value={seatsPerTable}
          onChange={e => setSeatsPerTable(Math.max(3, Math.min(12, Number(e.target.value))))} className="w-16 h-8
text-center" />
      </label>
      <label className="flex items-center gap-2 text-sm text-muted-foreground">
        桌数
        <Input type="number" min={1} max={20} value={tableCount}
          onChange={e => setTableCount(Math.max(1, Math.min(20, Number(e.target.value))))} className="w-16 h-8
text-center" />
      </label>
      <label className="flex items-center gap-2 text-sm text-muted-foreground">
        模式
        <select
          value={mode}
          onChange={e => setMode(e.target.value as SmartSeatMode)}
          className="h-8 px-2 rounded-md border border-input bg-background text-foreground text-sm"
        >
          <option value="tableRoundRobin">每桌轮转</option>
          <option value="tableGrouped">每组一桌</option>
          <option value="verticalS">竖S桌序</option>
          <option value="horizontalS">横S桌序</option>
        </select>
      </label>
      {mode === 'tableGrouped' && (
        <label className="flex items-center gap-2 text-sm text-muted-foreground">
          组数
          <Input type="number" min={2} max={20} value={groupCount}
            onChange={e => setGroupCount(Math.max(2, Math.min(20, Number(e.target.value))))} className="w-16 h-8
text-center" />
        </label>
      )}
      <label className="flex items-center gap-2 text-sm text-muted-foreground">
        桌子间距
        <Input type="number" min={0} max={100} value={tableGap}
          onChange={e => setTableGap(Math.max(0, Math.min(100, Number(e.target.value))))} className="w-16 h-8
text-center" />
      </label>
      <Button variant="outline" onClick={() => setRefPositions(defaultRefPositions)}>
        重置参照物
      </Button>
      <div className="flex items-center gap-2 text-xs text-muted-foreground">
        <label className="flex items-center gap-1 cursor-pointer">
          <input type="checkbox" checked={refVisible.screen} onChange={() => toggleRefVisible('screen')}
```

```
className="accent-primary" /> 幕布
        </label>
        <label className="flex items-center gap-1 cursor-pointer">
          <input type="checkbox" checked={refVisible.podium} onChange={() => toggleRefVisible('podium')}
className="accent-primary" /> 讲台
        </label>
        <label className="flex items-center gap-1 cursor-pointer">
          <input type="checkbox" checked={refVisible.frontDoor} onChange={() => toggleRefVisible('frontDoor')}
className="accent-primary" /> 前门
        </label>
        <label className="flex items-center gap-1 cursor-pointer">
          <input type="checkbox" checked={refVisible.backDoor} onChange={() => toggleRefVisible('backDoor')}
className="accent-primary" /> 后门
        </label>
        <label className="flex items-center gap-1 cursor-pointer">
          <input type="checkbox" checked={refVisible.window} onChange={() => toggleRefVisible('window')}
className="accent-primary" /> 窗
        </label>
        <label className="flex items-center gap-1 cursor-pointer">
          <input type="checkbox" checked={refLocked} onChange={e => setRefLocked(e.target.checked)}
className="accent-primary" /> 锁定参照物
        </label>
      </div>
      {assignment.length > 0 && <ExportButtons targetRef={printRef} filename="智能教室座位" />}
      {assignment.length > 0 && (
        <Button variant="outline" onClick={() => setCheckinOpen(true)} className="gap-2">
          <QrCode className="w-4 h-4" /> 签到
        </Button>
      )}
      <div className="flex gap-2 ml-auto">
        <Button variant="outline" onClick={() => autoSeat(true)} className="gap-2">
          <Shuffle className="w-4 h-4" /> 随机排座
        </Button>
        <Button onClick={() => autoSeat(false)} className="gap-2">
          <LayoutGrid className="w-4 h-4" /> 自动排座
        </Button>
      </div>
    </div>

    <div ref={printRef}>
      {assignment.length > 0 ? (
        <div className="flex justify-center overflow-auto">
          <div
            className="relative rounded-xl border border-border bg-card/40"
            style={{ width: roomWidth, height: roomHeight }}
          >
            {refVisible.screen && (
              <div
                className={refBadgeClass}
                style={{ left: refPositions.screen.x, top: refPositions.screen.y }}
                onMouseDown={e => startRefDrag(e, 'screen')}
              >
                <span className={refIconClass}></span>
                <span className={refTextClass}>幕布</span>
```

```
          </div>
        )}

        {refVisible.podium && (
          <div
            className={refBadgeClass}
            style={{ left: refPositions.podium.x, top: refPositions.podium.y }}
            onMouseDown={e => startRefDrag(e, 'podium')}
          >
            <span className={refIconClass}></span>
            <span className={refTextClass}>讲台</span>
          </div>
        )}

        {refVisible.frontDoor && (
          <div
            className={refBadgeClass}
            style={{ left: refPositions.frontDoor.x, top: refPositions.frontDoor.y }}
            onMouseDown={e => startRefDrag(e, 'frontDoor')}
          >
            <span className={refIconClass}></span>
            <span className={refTextClass}>前门</span>
          </div>
        )}

        {refVisible.backDoor && (
          <div
            className={refBadgeClass}
            style={{ left: refPositions.backDoor.x, top: refPositions.backDoor.y }}
            onMouseDown={e => startRefDrag(e, 'backDoor')}
          >
            <span className={refIconClass}></span>
            <span className={refTextClass}>后门</span>
          </div>
        )}

        {refVisible.window && (
          <div
            className={refBadgeClass}
            style={{ left: refPositions.window.x, top: refPositions.window.y }}
            onMouseDown={e => startRefDrag(e, 'window')}
          >
            <span className={refIconClass}></span>
            <span className={refTextClass}>窗</span>
          </div>
        )}

        <div className="absolute inset-0 flex items-center justify-center pointer-events-none">
          <div className="inline-grid pointer-events-auto" style={{ gridTemplateColumns: `repeat(${tableCols}, 1fr)`,
gap: `${tableGap}px` }}>
            {assignment.map((people, i) => renderRoundTable(i, people))}
          </div>
        </div>
      </div>
```

```
        </div>
      ) : (
        <div className="text-center py-20 text-muted-foreground">
          <p className="text-lg mb-2">点击「自动排座」开始安排</p>
          <p className="text-sm">圆形桌智能教室，每桌 {seatsPerTable} 人，共 {tableCount} 桌</p>
        </div>
      )}
    </div>

    {assignment.length > 0 && (
      <p className="text-center text-xs text-muted-foreground mt-4">
        拖拽姓名可交换座位；点击空座位可关闭/开放使用；幕布/讲台/前后门/窗支持显隐与拖拽
      </p>
    )}
    <SeatCheckinDialog
      open={checkinOpen}
      onOpenChange={setCheckinOpen}
      seatData={assignment}
      studentNames={students.map(s => s.name)}
      sceneType="smartClassroom"
      sceneConfig={{ seatsPerTable, tableCount, tableCols: Math.ceil(Math.sqrt(tableCount)) }}
    />
  </div>
 );
}
```

文件: src/components/seating/ConferenceRoom.tsx

```
import { useState, useRef, useEffect, useMemo, type MouseEvent as ReactMouseEvent } from 'react';
import { Input } from '@/components/ui/input';
import { Button } from '@/components/ui/button';
import { LayoutGrid, Shuffle, QrCode } from 'lucide-react';
import ExportButtons from '@/components/ExportButtons';
import SeatCheckinDialog from '@/components/SeatCheckinDialog';

interface Props {
  students: { id: string; name: string }[];
}

type ConferenceSeatMode = 'balanced' | 'groupCluster' | 'verticalS' | 'horizontalS';
type RefKey = 'screen' | 'podium' | 'window' | 'frontDoor' | 'backDoor';
type RefPositions = Record<RefKey, { x: number; y: number }>;
type RefVisible = Record<RefKey, boolean>;

type ConferenceAssignment = {
  headLeft: string;
  headRight: string;
  mainTop: string[];
  mainBottom: string[];
  companionTop: string[][];
  companionBottom: string[][];
};
```

```
function splitIntoGroups(names: string[], count: number) {
  const groups: string[][] = Array.from({ length: count }, () => []);
  names.forEach((n, i) => groups[i % count].push(n));
  return groups;
}

function buildDefaultRefPositions(roomWidth: number, roomHeight: number): RefPositions {
  const badgeW = 94;
  const centerX = Math.round((roomWidth - badgeW) / 2);
  const rightX = Math.max(24, roomWidth - badgeW - 24);
  const midY = Math.max(20, Math.round((roomHeight - 32) / 2));
  return {
    screen: { x: centerX, y: 22 },
    podium: { x: centerX, y: 74 },
    window: { x: 24, y: midY },
    frontDoor: { x: rightX, y: 120 },
    backDoor: { x: rightX, y: Math.max(160, roomHeight - 56) },
  };
}

export default function ConferenceRoom({ students }: Props) {
  const [seatsPerSide, setSeatsPerSide] = useState(10);
  const [groupCount, setGroupCount] = useState(4);
  const [mode, setMode] = useState<ConferenceSeatMode>('balanced');
  const [seatGap, setSeatGap] = useState(6);
  const [showCompanionSeats, setShowCompanionSeats] = useState(true);
  const [companionRows, setCompanionRows] = useState(1);
  const [assignment, setAssignment] = useState<ConferenceAssignment>({
    headLeft: '',
    headRight: '',
    mainTop: Array.from({ length: 10 }, () => ''),
    mainBottom: Array.from({ length: 10 }, () => ''),
    companionTop: [Array.from({ length: 10 }, () => '')],
    companionBottom: [Array.from({ length: 10 }, () => '')],
  });
  const [closedSeats, setClosedSeats] = useState<Set<string>>(new Set());
  const [dragFrom, setDragFrom] = useState<string | null>(null);
  const [dropTarget, setDropTarget] = useState<string | null>(null);
  const [seated, setSeated] = useState(false);
  const [checkinOpen, setCheckinOpen] = useState(false);

  const [refVisible, setRefVisible] = useState<RefVisible>({
    screen: true,
    podium: true,
    window: true,
    frontDoor: true,
    backDoor: true,
  });
  const [refLocked, setRefLocked] = useState(false);

  const printRef = useRef<HTMLDivElement>(null);
  const refDraggingRef = useRef<{ key: RefKey; startX: number; startY: number; origX: number; origY: number } |
null>(null);
```

```
  const seatW = 64;
  const seatH = 40;
  const gap = seatGap;
  const tableH = 64;
  const tableW = seatsPerSide * (seatW + gap) + gap;
  const headGap = 12;
  const sideGap = 10;

  const contentWidth = tableW + 2 * (seatW + headGap);
  const contentHeight =
    tableH +
    2 * (seatH + sideGap) +
    (showCompanionSeats ? 2 * companionRows * (seatH + sideGap) : 0);

  const roomWidth = Math.max(920, contentWidth + 160);
  const roomHeight = Math.max(640, contentHeight + 220);

  const tableX = (roomWidth - tableW) / 2;
  const tableY = (roomHeight - tableH) / 2;

  const defaultRefPositions = useMemo(
    () => buildDefaultRefPositions(roomWidth, roomHeight),
    [roomWidth, roomHeight]
  );

  const [refPositions, setRefPositions] = useState<RefPositions>(() =>
    buildDefaultRefPositions(920, 640)
  );

  const refBadgeClass =
    'absolute h-8 pl-2 pr-2.5 rounded-lg border border-primary/30 bg-primary/10 text-primary shadow-sm cursor-move
select-none inline-flex items-center gap-1.5';
  const refIconClass =
    'inline-flex items-center justify-center w-5 h-5 rounded-md border border-primary/30 bg-background/80 text-[11px]
leading-none';
  const refTextClass = 'text-[11px] font-medium leading-none tracking-wide';

  const sideOrderSlots = useMemo(() => {
    const slots: { side: 'top' | 'bottom'; index: number }[] = [];

    if (mode === 'verticalS') {
      for (let c = 0; c < seatsPerSide; c++) {
        if (c % 2 === 0) {
          slots.push({ side: 'top', index: c });
          slots.push({ side: 'bottom', index: c });
        } else {
          slots.push({ side: 'bottom', index: c });
          slots.push({ side: 'top', index: c });
        }
      }
      return slots;
    }
```

```
  if (mode === 'horizontalS' || mode === 'groupCluster') {
    for (let c = 0; c < seatsPerSide; c++) slots.push({ side: 'top', index: c });
    for (let c = seatsPerSide - 1; c >= 0; c--) slots.push({ side: 'bottom', index: c });
    return slots;
  }

  for (let c = 0; c < seatsPerSide; c++) slots.push({ side: 'top', index: c });
  for (let c = 0; c < seatsPerSide; c++) slots.push({ side: 'bottom', index: c });
  return slots;
}, [mode, seatsPerSide]);

const allSlots = useMemo(() => {
  const slots: string[] = ['head-left', 'head-right'];

  sideOrderSlots.forEach(({ side, index }) => {
    slots.push(`main-${side}-${index}`);
  });

  if (showCompanionSeats) {
    for (let r = 0; r < companionRows; r++) {
      sideOrderSlots.forEach(({ side, index }) => {
        slots.push(`companion-${side}-${r}-${index}`);
      });
    }
  }

  return slots;
}, [companionRows, showCompanionSeats, sideOrderSlots]);

useEffect(() => {
  setRefPositions(defaultRefPositions);
}, [defaultRefPositions]);

useEffect(() => {
  setAssignment(prev => ({
    headLeft: prev.headLeft,
    headRight: prev.headRight,
    mainTop: Array.from({ length: seatsPerSide }, (_, i) => prev.mainTop[i] || ''),
    mainBottom: Array.from({ length: seatsPerSide }, (_, i) => prev.mainBottom[i] || ''),
    companionTop: Array.from({ length: companionRows }, (_, r) =>
      Array.from({ length: seatsPerSide }, (_, i) => prev.companionTop[r]?.[i] || '')
    ),
    companionBottom: Array.from({ length: companionRows }, (_, r) =>
      Array.from({ length: seatsPerSide }, (_, i) => prev.companionBottom[r]?.[i] || '')
    ),
  }));
}, [seatsPerSide, companionRows]);

useEffect(() => {
  setClosedSeats(prev => {
    const next = new Set<string>();
    allSlots.forEach(slot => {
      if (prev.has(slot)) next.add(slot);
    });
```

```
    return next;
  });
}, [allSlots]);

useEffect(() => {
  const handleMouseMove = (e: MouseEvent) => {
    if (!refDraggingRef.current) return;
    const dx = e.clientX - refDraggingRef.current.startX;
    const dy = e.clientY - refDraggingRef.current.startY;
    const key = refDraggingRef.current.key;
    setRefPositions(prev => ({
      ...prev,
      [key]: {
        x: refDraggingRef.current!.origX + dx,
        y: refDraggingRef.current!.origY + dy,
      },
    }));
  };

  const handleMouseUp = () => {
    refDraggingRef.current = null;
  };

  window.addEventListener('mousemove', handleMouseMove);
  window.addEventListener('mouseup', handleMouseUp);
  return () => {
    window.removeEventListener('mousemove', handleMouseMove);
    window.removeEventListener('mouseup', handleMouseUp);
  };
}, []);

const toggleRefVisible = (key: RefKey) => {
  setRefVisible(prev => ({ ...prev, [key]: !prev[key] }));
};

const startRefDrag = (e: ReactMouseEvent, key: RefKey) => {
  if (refLocked) return;
  e.preventDefault();
  e.stopPropagation();
  refDraggingRef.current = {
    key,
    startX: e.clientX,
    startY: e.clientY,
    origX: refPositions[key].x,
    origY: refPositions[key].y,
  };
};

const getSeatValue = (data: ConferenceAssignment, slot: string) => {
  if (slot === 'head-left') return data.headLeft;
  if (slot === 'head-right') return data.headRight;

  const parts = slot.split('-');
  if (parts[0] === 'main') {
```

```
    const side = parts[1] as 'top' | 'bottom';
    const index = Number(parts[2]);
    return side === 'top' ? data.mainTop[index] || '' : data.mainBottom[index] || '';
  }

  const side = parts[1] as 'top' | 'bottom';
  const row = Number(parts[2]);
  const index = Number(parts[3]);
  return side === 'top'
    ? data.companionTop[row]?.[index] || ''
    : data.companionBottom[row]?.[index] || '';
};

const setSeatValue = (data: ConferenceAssignment, slot: string, value: string) => {
  if (slot === 'head-left') {
    data.headLeft = value;
    return;
  }
  if (slot === 'head-right') {
    data.headRight = value;
    return;
  }

  const parts = slot.split('-');
  if (parts[0] === 'main') {
    const side = parts[1] as 'top' | 'bottom';
    const index = Number(parts[2]);
    if (side === 'top') data.mainTop[index] = value;
    else data.mainBottom[index] = value;
    return;
  }

  const side = parts[1] as 'top' | 'bottom';
  const row = Number(parts[2]);
  const index = Number(parts[3]);
  if (side === 'top') data.companionTop[row][index] = value;
  else data.companionBottom[row][index] = value;
};

const toggleSeatOpen = (slot: string) => {
  setClosedSeats(prev => {
    const next = new Set(prev);
    if (next.has(slot)) next.delete(slot);
    else next.add(slot);
    return next;
  });
};

const autoSeat = (shuffle = false) => {
  const names = shuffle
    ? [...students.map(s => s.name)].sort(() => Math.random() - 0.5)
    : students.map(s => s.name);

  const next: ConferenceAssignment = {
```

```
    headLeft: '',
    headRight: '',
    mainTop: Array.from({ length: seatsPerSide }, () => ''),
    mainBottom: Array.from({ length: seatsPerSide }, () => ''),
    companionTop: Array.from({ length: companionRows }, () => Array.from({ length: seatsPerSide }, () => '')),
    companionBottom: Array.from({ length: companionRows }, () => Array.from({ length: seatsPerSide }, () => '')),
  };

  const availableSlots = allSlots.filter(slot => !closedSeats.has(slot));

  if (mode === 'groupCluster') {
    const groups = splitIntoGroups(names, Math.max(1, groupCount));
    let cursor = 0;
    groups.forEach(group => {
      group.forEach(n => {
        if (cursor >= availableSlots.length) return;
        setSeatValue(next, availableSlots[cursor++], n);
      });
    });
  } else {
    names.slice(0, availableSlots.length).forEach((name, i) => {
      setSeatValue(next, availableSlots[i], name);
    });
  }

  setAssignment(next);
  setSeated(true);
};

const renderSeat = (x: number, y: number, name: string, slot: string) => {
  const isClosed = closedSeats.has(slot);
  const isDragging = dragFrom === slot;
  const isOver = dropTarget === slot;

  return (
    <g
      key={slot}
      style={{ cursor: name && !isClosed ? 'grab' : 'pointer' }}
      onMouseDown={
        name && !isClosed
          ? e => {
              e.stopPropagation();
              setDragFrom(slot);
              setDropTarget(slot);
            }
          : undefined
      }
      onMouseEnter={() => {
        if (dragFrom && !isClosed) setDropTarget(slot);
      }}
      onMouseUp={() => {
        if (!dragFrom || !dropTarget) return;
        const from = dragFrom;
        const to = dropTarget;
```

```
    if (from === to || closedSeats.has(from) || closedSeats.has(to)) {
      setDragFrom(null);
      setDropTarget(null);
      return;
    }

    setAssignment(prev => {
      const next: ConferenceAssignment = {
        headLeft: prev.headLeft,
        headRight: prev.headRight,
        mainTop: [...prev.mainTop],
        mainBottom: [...prev.mainBottom],
        companionTop: prev.companionTop.map(row => [...row]),
        companionBottom: prev.companionBottom.map(row => [...row]),
      };

      const fromVal = getSeatValue(next, from);
      const toVal = getSeatValue(next, to);
      setSeatValue(next, from, toVal);
      setSeatValue(next, to, fromVal);
      return next;
    });

    setDragFrom(null);
    setDropTarget(null);
  }}
  onClick={() => {
    if (!name) toggleSeatOpen(slot);
  }}
>
  <rect
    x={x}
    y={y}
    width={seatW}
    height={seatH}
    rx={6}
    className={
      isClosed
        ? 'fill-muted stroke-destructive/60'
        : isDragging
          ? 'fill-primary/20 stroke-primary'
          : isOver
            ? 'fill-accent stroke-primary'
            : name
              ? 'fill-card stroke-border'
              : 'fill-muted/50 stroke-border/50'
    }
    strokeWidth={isOver ? 2.5 : 1.5}
  />
  {isClosed && (
    <text
      x={x + seatW / 2}
      y={y + seatH / 2 + 1}
```

```
    .select('*')
    .eq('id', sessionId)
    .single()
    .then(({ data, error }) => {
      if (error || !data) {
        setSessionValid(false);
      } else if ((data as any).status !== 'active') {
        setSessionValid(false);
        setStatus('expired');
      } else {
        setSessionValid(true);
        const names = (data as any).student_names;
        if (Array.isArray(names)) {
          setStudentNames(names);
        }
      }
    });
}, [sessionId]);

useEffect(() => {
  const handler = (e: MouseEvent) => {
    if (
      suggestionsRef.current &&
      !suggestionsRef.current.contains(e.target as Node) &&
      inputRef.current &&
      !inputRef.current.contains(e.target as Node)
    ) {
      setShowSuggestions(false);
    }
  };
  document.addEventListener('mousedown', handler);
  return () => document.removeEventListener('mousedown', handler);
}, []);

const handleNameChange = (val: string) => {
  setName(val);
  if (val.trim() && studentNames.length > 0) {
    const filtered = studentNames.filter(n => n.includes(val.trim()));
    setSuggestions(filtered.slice(0, 8));
    setShowSuggestions(filtered.length > 0);
  } else {
    setSuggestions([]);
    setShowSuggestions(false);
  }
};

const selectSuggestion = (s: string) => {
  setName(s);
  setShowSuggestions(false);
  setSuggestions([]);
};

const handleCheckin = async () => {
  const trimmed = name.trim();
```

```
  if (!trimmed || !sessionId) return;
  setStatus('loading');
  const { data: existing } = await supabase
    .from('checkin_records')
    .select('id')
    .eq('session_id', sessionId)
    .eq('student_name', trimmed)
    .maybeSingle();
  if (existing) {
    setStatus('success');
    return;
  }
  const { error } = await supabase
    .from('checkin_records')
    .insert({ session_id: sessionId, student_name: trimmed, status: 'pending' });
  if (error) {
    setStatus('error');
    setErrorMsg(t('checkinPage.failed'));
    return;
  }
  setStatus('success');
};

if (sessionValid === null) {
  return (
    <div className="min-h-screen flex items-center justify-center bg-background">
      <div className="text-muted-foreground">{t('checkinPage.loading')}</div>
    </div>
  );
}

if (!sessionValid || status === 'expired') {
  return (
    <div className="min-h-screen flex items-center justify-center bg-background p-4">
      <div className="text-center space-y-4">
        <Clock className="w-12 h-12 text-muted-foreground mx-auto" />
        <h1 className="text-xl font-bold text-foreground">{t('checkinPage.expired')}</h1>
        <p className="text-sm text-muted-foreground">{t('checkinPage.expiredDesc')}</p>
      </div>
    </div>
  );
}

if (status === 'success') {
  return (
    <div className="min-h-screen flex items-center justify-center bg-background p-4">
      <div className="text-center space-y-4">
        <CheckCircle2 className="w-16 h-16 text-primary mx-auto" />
        <h1 className="text-xl font-bold text-foreground">{t('checkinPage.success')}</h1>
        <p className="text-sm text-muted-foreground">{t('checkinPage.completed').replace('{0}', name.trim())}</p>
      </div>
    </div>
  );
}
```

```
  return (
    <div className="min-h-screen flex items-center justify-center bg-background p-4">
      <div className="w-full max-w-sm space-y-6">
        <div className="text-center space-y-2">
          <div className="text-5xl"></div>
          <h1 className="text-xl font-bold text-foreground">{t('checkinPage.title')}</h1>
          <p className="text-sm text-muted-foreground">{t('checkinPage.desc')}</p>
        </div>

        <div className="space-y-3">
          <div className="relative">
            <Input
              ref={inputRef}
              value={name}
              onChange={e => handleNameChange(e.target.value)}
              onFocus={() => { if (suggestions.length > 0) setShowSuggestions(true); }}
              onKeyDown={e => { if (e.key === 'Enter') { setShowSuggestions(false); handleCheckin(); } }}
              placeholder={t('checkinPage.namePlaceholder')}
              className="h-12 text-center text-lg"
              autoFocus
            />
            {showSuggestions && suggestions.length > 0 && (
              <div ref={suggestionsRef} className="absolute z-50 w-full mt-1 rounded-md border border-border
bg-popover shadow-md overflow-hidden">
                {suggestions.map((s, i) => (
                  <button key={i} className="w-full text-left px-4 py-2.5 text-sm text-popover-foreground hover:bg-accent
hover:text-accent-foreground transition-colors" onMouseDown={(e) => { e.preventDefault(); selectSuggestion(s); }}>
                    {s}
                  </button>
                ))}
              </div>
            )}
          </div>
          <Button onClick={handleCheckin} disabled={!name.trim() || status === 'loading'} className="w-full h-12
text-base">
            {status === 'loading' ? t('checkinPage.checking') : t('checkinPage.checkinBtn')}
          </Button>
          {status === 'error' && (
            <div className="flex items-center justify-center gap-2 text-destructive text-sm">
              <XCircle className="w-4 h-4" />
              <span>{errorMsg}</span>
            </div>
          )}
        </div>
      </div>
    </div>
  );
}
```

文件: src/pages/DiscussPage.tsx

```
import { useState, useEffect } from 'react';
```

```
import { useParams } from 'react-router-dom';
import { useLanguage } from '@/contexts/LanguageContext';
import { supabase } from '@/integrations/supabase/client';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Send, User } from 'lucide-react';

export default function DiscussPage() {
  const { topicId } = useParams<{ topicId: string }>();
  const { t } = useLanguage();
  const [topic, setTopic] = useState<{ title: string } | null>(null);
  const [nickname, setNickname] = useState('');
  const [nicknameConfirmed, setNicknameConfirmed] = useState(false);
  const [content, setContent] = useState('');
  const [sending, setSending] = useState(false);
  const [sent, setSent] = useState(false);
  const [error, setError] = useState('');
  const [recentMessages, setRecentMessages] = useState<{ nickname: string; content: string }[]>([]);

  useEffect(() => {
    if (!topicId) return;
    supabase
      .from('discussion_topics' as any)
      .select('title')
      .eq('id', topicId)
      .single()
      .then(({ data, error }) => {
        if (data) setTopic(data as any);
        if (error) setError(t('discuss.topicNotFound'));
      });

    supabase
      .from('barrage_messages' as any)
      .select('content, nickname')
      .eq('topic_id', topicId)
      .order('created_at', { ascending: false })
      .limit(5)
      .then(({ data }) => {
        if (data) setRecentMessages((data as any[]).map((m: any) => ({ nickname: m.nickname, content: m.content
})).reverse());
      });

    const saved = localStorage.getItem(`discuss-nick-${topicId}`);
    if (saved) {
      setNickname(saved);
      setNicknameConfirmed(true);
    }
  }, [topicId]);

  const handleConfirmNickname = () => {
    if (!nickname.trim()) return;
    setNicknameConfirmed(true);
    localStorage.setItem(`discuss-nick-${topicId}`, nickname.trim());
  };
```

```
const handleSend = async () => {
  if (!content.trim() || !topicId) return;
  setSending(true);
  const { error } = await supabase
    .from('barrage_messages' as any)
    .insert({ topic_id: topicId, content: content.trim(), nickname: nickname.trim() } as any);
  setSending(false);
  if (error) {
    setError(t('discuss.sendFailed'));
  } else {
    setRecentMessages(prev => [...prev.slice(-4), { nickname: nickname.trim(), content: content.trim() }]);
    setContent('');
    setSent(true);
    setTimeout(() => setSent(false), 2000);
  }
};

if (error && !topic) {
  return (
    <div className="min-h-screen bg-background flex items-center justify-center p-4">
      <div className="text-center text-muted-foreground">{error}</div>
    </div>
  );
}

if (!nicknameConfirmed) {
  return (
    <div className="min-h-screen bg-background flex flex-col items-center justify-center p-4">
      <div className="w-full max-w-sm space-y-6 text-center">
        <div>
          <div className="text-4xl mb-3"></div>
          <h1 className="text-xl font-bold text-foreground">{topic?.title || t('discuss.loading')}</h1>
          <p className="text-sm text-muted-foreground mt-1">{t('discuss.enterNickname')}</p>
        </div>
        <div className="space-y-3">
          <div className="relative">
            <User className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-muted-foreground" />
            <Input
              value={nickname}
              onChange={e => setNickname(e.target.value.slice(0, 12))}
              placeholder={t('discuss.nicknamePlaceholder')}
              className="pl-9"
              maxLength={12}
              onKeyDown={e => e.key === 'Enter' && handleConfirmNickname()}
            />
          </div>
          <Button onClick={handleConfirmNickname} disabled={!nickname.trim()} className="w-full gap-2">
            {t('discuss.enterDiscussion')}
          </Button>
        </div>
        <div className="text-xs text-muted-foreground">{t('discuss.noRegister')}</div>
      </div>
    </div>
```

```
    );
  }

  return (
    <div className="min-h-screen bg-background flex flex-col items-center justify-center p-4">
      <div className="w-full max-w-md space-y-6">
        <div className="text-center">
          <div className="text-sm text-muted-foreground mb-1">{t('discuss.topic')}</div>
          <h1 className="text-xl font-bold text-foreground">{topic?.title || t('discuss.loading')}</h1>
          <div className="text-xs text-muted-foreground mt-1">
            {t('discuss.nickname')}: <span className="text-foreground font-medium">{nickname}</span>
            <button onClick={() => setNicknameConfirmed(false)} className="ml-2 underline
text-primary">{t('discuss.edit')}</button>
          </div>
        </div>

        {recentMessages.length > 0 && (
          <div className="bg-muted/50 rounded-xl p-3 space-y-1.5">
            <div className="text-xs text-muted-foreground mb-1">{t('discuss.recentBarrage')}</div>
            {recentMessages.map((msg, i) => (
              <div key={i} className="text-sm text-foreground bg-card rounded-lg px-3 py-1.5">
                <span className="text-xs text-muted-foreground mr-1.5">{msg.nickname}:</span>
                {msg.content}
              </div>
            ))}
          </div>
        )}

        <div className="space-y-3">
          <textarea
            value={content}
            onChange={e => setContent(e.target.value.slice(0, 50))}
            placeholder={t('discuss.inputBarrage')}
            className="w-full h-24 rounded-xl border border-border bg-card px-4 py-3 text-base text-foreground
placeholder:text-muted-foreground focus:outline-none focus:ring-2 focus:ring-primary resize-none"
            maxLength={50}
          />
          <div className="flex items-center justify-between">
            <span className="text-xs text-muted-foreground">{content.length}/50</span>
            <Button onClick={handleSend} disabled={!content.trim() || sending} className="gap-2 px-6">
              <Send className="w-4 h-4" />
              {sending ? t('discuss.sendingBtn') : t('discuss.send')}
            </Button>
          </div>
        </div>

        {sent && (
          <div className="text-center text-sm text-primary font-medium animate-pulse">{t('discuss.sendSuccess')}</div>
        )}
        {error && topic && (
          <div className="text-center text-sm text-destructive">{error}</div>
        )}
        <div className="text-center text-xs text-muted-foreground pt-4">{t('discuss.dataPrivacy')}</div>
      </div>
```

```
    </div>
  );
}
```

文件: src/pages/GoRedirect.tsx

```
import { useEffect, useState } from 'react';
import { useSearchParams } from 'react-router-dom';
import { useLanguage } from '@/contexts/LanguageContext';
import { ExternalLink, Copy, Check, Smartphone } from 'lucide-react';
import { Button } from '@/components/ui/button';

function isWeChat(): boolean {
  return /micromessenger/i.test(navigator.userAgent.toLowerCase());
}

export default function GoRedirect() {
  const [params] = useSearchParams();
  const { t } = useLanguage();
  const url = params.get('url') || '';
  const [copied, setCopied] = useState(false);

  useEffect(() => {
    if (url && !isWeChat()) {
      window.location.href = url;
    }
  }, [url]);

  const handleCopy = async () => {
    try {
      await navigator.clipboard.writeText(url);
      setCopied(true);
      setTimeout(() => setCopied(false), 2000);
    } catch {
      const ta = document.createElement('textarea');
      ta.value = url;
      ta.style.position = 'fixed';
      ta.style.opacity = '0';
      document.body.appendChild(ta);
      ta.select();
      document.execCommand('copy');
      document.body.removeChild(ta);
      setCopied(true);
      setTimeout(() => setCopied(false), 2000);
    }
  };

  if (!url) {
    return (
      <div className="min-h-screen flex items-center justify-center bg-background text-muted-foreground">
        {t('go.invalidLink')}
      </div>
    );
```

```
  }

  if (!isWeChat()) {
    return (
      <div className="min-h-screen flex items-center justify-center bg-background text-muted-foreground">
        {t('go.redirecting')}
      </div>
    );
  }

  return (
    <div className="min-h-screen bg-background flex flex-col items-center justify-center p-6">
      <div className="max-w-sm w-full space-y-6 text-center">
        <div className="inline-flex items-center justify-center w-16 h-16 rounded-full bg-primary/10 mx-auto">
          <Smartphone className="w-8 h-8 text-primary" />
        </div>

        <div className="space-y-2">
          <h1 className="text-xl font-bold text-foreground">{t('go.openInBrowser')}</h1>
          <p className="text-sm text-muted-foreground">{t('go.wechatWarning')}</p>
        </div>

        <div className="bg-card border border-border rounded-2xl p-4 text-left space-y-3">
          <div className="flex items-center gap-2 font-medium text-foreground text-sm">
            <span className="inline-flex items-center justify-center w-6 h-6 rounded-full bg-primary
text-primary-foreground text-xs font-bold">1</span>
            {t('go.recommended')}
          </div>
          <p className="text-sm text-muted-foreground pl-8">{t('go.recommendedDesc')}</p>
          <div className="flex justify-end pr-2">
            <div className="text-2xl animate-bounce">↗</div>
          </div>
        </div>

        <div className="bg-card border border-border rounded-2xl p-4 text-left space-y-3">
          <div className="flex items-center gap-2 font-medium text-foreground text-sm">
            <span className="inline-flex items-center justify-center w-6 h-6 rounded-full bg-secondary
text-secondary-foreground text-xs font-bold">2</span>
            {t('go.copyLink')}
          </div>
          <div className="bg-muted rounded-xl p-3 break-all text-xs text-muted-foreground font-mono">{url}</div>
          <Button onClick={handleCopy} variant="outline" className="w-full gap-2">
            {copied ? (
              <><Check className="w-4 h-4" />{t('go.copied')}</>
            ) : (
              <><Copy className="w-4 h-4" />{t('go.copyLink')}</>
            )}
          </Button>
        </div>

        <a href={url} target="_blank" rel="noopener noreferrer" className="inline-flex items-center gap-1 text-sm
text-primary hover:underline">
          <ExternalLink className="w-3.5 h-3.5" />
          {t('go.tryDirect')}
```

```
      </a>
    </div>
  </div>
 );
}
```

文件: src/pages/Index.tsx

```
import { useEffect, useRef, useState } from 'react';
import { StudentProvider } from '@/contexts/StudentContext';
import { ThemeProvider } from '@/contexts/ThemeContext';
import { useAuth } from '@/contexts/AuthContext';
import { useLanguage } from '@/contexts/LanguageContext';
import StudentSidebar from '@/components/StudentSidebar';
import TabNavigation, { TabId } from '@/components/TabNavigation';
import RandomPicker from '@/components/RandomPicker';
import TeamworkPanel from '@/components/TeamworkPanel';
import SeatChart from '@/components/SeatChart';
import ToolkitPanel from '@/components/ToolkitPanel';
import BoardPanel from '@/components/BoardPanel';
import QuizPanel from '@/components/QuizPanel';
import AchievementPanel from '@/components/AchievementPanel';
import SettingsPanel from '@/components/SettingsPanel';
import WeChatBanner from '@/components/WeChatBanner';
import ClassLibrary from '@/components/ClassLibrary';
import LanguageSelector from '@/components/LanguageSelector';
import StoryboardPanel from '@/components/StoryboardPanel';
import PPTPanel from '@/components/PPTPanel';
import VisualizationPanel from '@/components/VisualizationPanel';
import { LogIn } from 'lucide-react';
import { useNavigate } from 'react-router-dom';

const Index = () => {
  const { user, approvalStatus, isAdmin } = useAuth();
  const { t } = useLanguage();
  const isApproved = user && approvalStatus === 'approved';
  const navigate = useNavigate();
  const [activeTab, setActiveTab] = useState<TabId>('random');
  const [sidebarOpen, setSidebarOpen] = useState(false);
  const [sidebarCollapsed, setSidebarCollapsed] = useState(false);
  const [sidebarMode, setSidebarMode] = useState<'list' | 'library'>('list');
  const [sidebarModeTransitioning, setSidebarModeTransitioning] = useState(false);
  const [sidebarModeEntering, setSidebarModeEntering] = useState(false);
  const [sidebarTransitionDirection, setSidebarTransitionDirection] = useState<'to-library' | 'to-list'>('to-library');
  const modeTimerRef = useRef<number | null>(null);
  const enterFrameRef = useRef<number | null>(null);

  useEffect(() => {
    return () => {
      if (modeTimerRef.current !== null) {
        window.clearTimeout(modeTimerRef.current);
      }
      if (enterFrameRef.current !== null) {
```

```
      window.cancelAnimationFrame(enterFrameRef.current);
    }
  };
}, []);

const switchSidebarMode = (nextMode: 'list' | 'library') => {
  if (sidebarMode === nextMode) return;

  if (modeTimerRef.current !== null) {
    window.clearTimeout(modeTimerRef.current);
  }
  if (enterFrameRef.current !== null) {
    window.cancelAnimationFrame(enterFrameRef.current);
  }

  setSidebarTransitionDirection(nextMode === 'library' ? 'to-library' : 'to-list');

  setSidebarModeTransitioning(true);
  modeTimerRef.current = window.setTimeout(() => {
    setSidebarMode(nextMode);
    setSidebarModeEntering(true);

    enterFrameRef.current = window.requestAnimationFrame(() => {
      setSidebarModeEntering(false);
      enterFrameRef.current = null;
    });

    setSidebarModeTransitioning(false);
    modeTimerRef.current = null;
  }, 150);
};

const handleTabChange = (tab: TabId) => {
  setActiveTab(tab);
  // Keep collapse state stable across tabs; only normalize mode for consistency.
  setSidebarMode('list');
  setSidebarModeTransitioning(false);
  setSidebarModeEntering(false);
  setSidebarOpen(false);
};

const renderContent = () => {
  switch (activeTab) {
    case 'random': return <RandomPicker />;
    case 'teamwork': return <TeamworkPanel />;
    case 'seats': return <SeatChart />;
    case 'board': return <BoardPanel />;
    case 'quiz': return <QuizPanel />;
    case 'sketch': return <StoryboardPanel />;
    case 'ppt': return <PPTPanel />;
    case 'visual': return <VisualizationPanel />;
    case 'achieve': return <AchievementPanel />;
    case 'toolkit': return <ToolkitPanel />;
  }
```

```
  };
  return (
    <ThemeProvider>
      <StudentProvider>
        <div className="flex flex-col h-screen bg-surface overflow-hidden">
          <WeChatBanner />

          {/* Header */}
          <header className="flex items-center justify-between px-3 sm:px-4 py-0 bg-card border-b border-border">
            <div className="flex items-center gap-2 sm:gap-3">
              <img src="/logo.png" alt="教创搭子" className="h-14 w-auto transition-[filter] duration-300" style={{ filter:
'hue-rotate(var(--logo-hue-rotate, 0deg))' }} />
              <span className="text-xs sm:text-sm text-muted-foreground font-light hidden
sm:inline">{t('app.subtitle')}</span>
            </div>
            <div className="flex items-center gap-1">
              {/* Mobile sidebar toggle */}
              <button
                onClick={() => setSidebarOpen(!sidebarOpen)}
                className="lg:hidden p-2 rounded-lg hover:bg-muted transition-colors text-muted-foreground"
              >
                <span class="text-base"></span>
              </button>
              {/* Auth button */}
              {isAdmin && (
                <button
                  onClick={() => navigate('/admin')}
                  className="p-2 rounded-lg hover:bg-muted transition-colors text-muted-foreground"
                  title={t('header.admin')}
                >

                </button>
              )}
              {user ? (
                <span class="text-xs text-muted-foreground hidden sm:inline mr-1 truncate max-w-[120px]">
                  {user.email}
                </span>
              ) : (
                <button
                  onClick={() => navigate('/auth')}
                  className="p-2 rounded-lg hover:bg-muted transition-colors text-muted-foreground"
                  title={t('header.login')}
                >
                  <LogIn class="w-5 h-5" />
                </button>
              )}
              <SettingsPanel />
              <LanguageSelector />
            </div>
          </header>

          {/* Tab Navigation */}
          <TabNavigation activeTab={activeTab} onTabChange={handleTabChange} isLoggedIn={!!isApproved}
userEmail={user?.email} />
```

```
        {/* Main Content */}
        <div className="flex flex-1 overflow-hidden relative">
          {/* Mobile sidebar overlay */}
          {sidebarOpen && (
            <div className="fixed inset-0 z-40 bg-foreground/20 backdrop-blur-sm lg:hidden" onClick={() =>
setSidebarOpen(false)} />
          )}

          {/* Sidebar area */}
          {true ? (
            <div className={`
              fixed lg:relative z-50 lg:z-auto h-full min-h-0
              transition-transform duration-300 ease-in-out
              ${sidebarOpen ? 'translate-x-0' : '-translate-x-full lg:translate-x-0'}
            `}>
              <div
                className={` h-full min-h-0 flex flex-col transition-all duration-150 ease-out will-change-transform ${
                  sidebarModeTransitioning
                    ? sidebarTransitionDirection === 'to-library'
                      ? 'opacity-0 -translate-x-2'
                      : 'opacity-0 translate-x-2'
                    : sidebarModeEntering
                      ? sidebarTransitionDirection === 'to-library'
                        ? 'opacity-0 translate-x-2'
                        : 'opacity-0 -translate-x-2'
                      : 'opacity-100 translate-x-0'
                }`}
              >
                {isApproved && sidebarMode === 'library' ? (
                  <div className="h-full min-h-0 flex flex-col w-[500px] lg:w-[560px]">
                    <ClassLibrary onBackToList={() => switchSidebarMode('list')} />
                  </div>
                ) : (
                  <StudentSidebar
                    onClose={() => setSidebarOpen(false)}
                    collapsed={sidebarCollapsed}
                    onToggleCollapse={() => setSidebarCollapsed(c => !c)}
                    onOpenLibrary={isApproved ? () => switchSidebarMode('library') : undefined}
                  />
                )}
              </div>
            </div>
          ) : null}

          <div className="flex-1 min-w-0 overflow-hidden transition-[width] duration-150 ease-out">
            {renderContent()}
          </div>
        </div>
      </div>
    </StudentProvider>
  </ThemeProvider>
 );
};
```

```
export default Index;
```

文件: src/pages/NotFound.tsx

```
import { useLocation } from "react-router-dom";
import { useEffect } from "react";
import { useLanguage } from '@/contexts/LanguageContext';

const NotFound = () => {
  const location = useLocation();
  const { t } = useLanguage();

  useEffect(() => {
    console.error("404 Error: User attempted to access non-existent route:", location.pathname);
  }, [location.pathname]);

  return (
    <div className="flex min-h-screen items-center justify-center bg-muted">
      <div className="text-center">
        <h1 className="mb-4 text-4xl font-bold">404</h1>
        <p className="mb-4 text-xl text-muted-foreground">{t('notFound.title')}</p>
        <a href="/" className="text-primary underline hover:text-primary/90">
          {t('notFound.back')}
        </a>
      </div>
    </div>
  );
};

export default NotFound;
```

文件: src/pages/PollVotePage.tsx

```
import { useState, useEffect } from 'react';
import { useParams } from 'react-router-dom';
import { useLanguage } from '@/contexts/LanguageContext';
import { supabase } from '@/integrations/supabase/client';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { toast } from '@/hooks/use-toast';
import { CheckCircle2, Lock, User, Vote } from 'lucide-react';

interface PollOption {
  label: string;
  color: string;
}

interface Poll {
  id: string;
  title: string;
  poll_type: 'single' | 'multiple';
```

```
  options: PollOption[];
  status: string;
}

export default function PollVotePage() {
  const { pollId } = useParams<{ pollId: string }>();
  const { t } = useLanguage();
  const [poll, setPoll] = useState<Poll | null>(null);
  const [loading, setLoading] = useState(true);
  const [nickname, setNickname] = useState('');
  const [nicknameConfirmed, setNicknameConfirmed] = useState(false);
  const [selected, setSelected] = useState<number[]>([]);
  const [submitting, setSubmitting] = useState(false);
  const [submitted, setSubmitted] = useState(false);

  useEffect(() => {
    if (!pollId) return;
    supabase.from('polls').select('*').eq('id', pollId).single()
      .then(({ data }) => {
        if (data) setPoll(data as any);
        setLoading(false);
      });

    const saved = localStorage.getItem(`poll-nick-${pollId}`);
    if (saved) {
      setNickname(saved);
      setNicknameConfirmed(true);
    }
  }, [pollId]);

  const confirmNickname = () => {
    if (!nickname.trim()) return;
    setNicknameConfirmed(true);
    localStorage.setItem(`poll-nick-${pollId}`, nickname.trim());
  };

  const toggleOption = (idx: number) => {
    if (!poll) return;
    if (poll.poll_type === 'single') {
      setSelected([idx]);
    } else {
      setSelected(prev => prev.includes(idx) ? prev.filter(i => i !== idx) : [...prev, idx]);
    }
  };

  const handleSubmit = async () => {
    if (!pollId || !poll || selected.length === 0) return;

    // Check if already voted
    const voterToken = localStorage.getItem('poll-voter-token') || crypto.randomUUID();
    localStorage.setItem('poll-voter-token', voterToken);

    setSubmitting(true);
    const { error } = await supabase.from('poll_votes').insert({
```

```
    poll_id: pollId,
    voter_token: voterToken,
    voter_name: nickname.trim() || t('board.anonymous'),
    selected_options: selected,
  } as any);
  setSubmitting(false);

  if (error) {
    toast({ title: error.message, variant: 'destructive' });
  } else {
    setSubmitted(true);
    toast({ title: t('poll.voteSuccess') });
  }
};

if (loading) return <div className="min-h-screen flex items-center justify-center
text-muted-foreground">{t('common.loading')}</div>;
if (!poll) return <div className="min-h-screen flex items-center justify-center
text-muted-foreground">{t('poll.notFound')}</div>;

if (poll.status !== 'active') {
  return (
    <div className="min-h-screen flex items-center justify-center bg-background p-4">
      <div className="text-center space-y-4">
        <Lock className="w-12 h-12 text-muted-foreground mx-auto" />
        <h1 className="text-xl font-bold text-foreground">{t('poll.pollEnded')}</h1>
        <p className="text-sm text-muted-foreground">{t('poll.pollEndedMsg')}</p>
      </div>
    </div>
  );
}

if (submitted) {
  return (
    <div className="min-h-screen flex items-center justify-center bg-background p-4">
      <div className="text-center space-y-4">
        <CheckCircle2 className="w-16 h-16 text-primary mx-auto" />
        <h1 className="text-xl font-bold text-foreground">{t('poll.voteSuccess')}</h1>
        <p className="text-sm text-muted-foreground">{t('poll.thankYou')}</p>
      </div>
    </div>
  );
}

if (!nicknameConfirmed) {
  return (
    <div className="min-h-screen bg-background flex flex-col items-center justify-center p-4">
      <div className="w-full max-w-sm space-y-6 text-center">
        <div>
          <div className="text-4xl mb-3"></div>
          <h1 className="text-xl font-bold text-foreground">{poll.title}</h1>
          <p className="text-sm text-muted-foreground mt-1">{t('poll.joinPoll')}</p>
        </div>
        <div className="space-y-3">
```

```
          <div className="relative">
            <User className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-muted-foreground" />
            <Input
              value={nickname}
              onChange={e => setNickname(e.target.value.slice(0, 12))}
              placeholder={t('poll.namePlaceholder')}
              className="pl-9"
              maxLength={12}
              onKeyDown={e => e.key === 'Enter' && confirmNickname()}
              autoFocus
            />
          </div>
          <Button onClick={confirmNickname} disabled={!nickname.trim()} className="w-full gap-2">
            {t('poll.joinPoll')}
          </Button>
        </div>
      </div>
    </div>
  );
}

return (
  <div className="min-h-screen bg-background flex flex-col items-center justify-center p-4">
    <div className="w-full max-w-md space-y-6">
      <div className="text-center">
        <div className="text-sm text-muted-foreground mb-1"> {poll.title}</div>
        <h1 className="text-xl font-bold text-foreground">{t('poll.castYourVote')}</h1>
        <p className="text-xs text-muted-foreground mt-1">
          {poll.poll_type === 'single' ? t('poll.selectOne') : t('poll.selectMultiple')}
        </p>
        <div className="text-xs text-muted-foreground mt-1">
          {t('board.nickname')}: <span className="text-foreground font-medium">{nickname}</span>
          <button onClick={() => setNicknameConfirmed(false)} className="ml-2 underline
text-primary">{t('common.edit')}</button>
        </div>
      </div>

      <div className="space-y-3">
        {(poll.options as PollOption[]).map((opt, idx) => (
          <button
            key={idx}
            onClick={() => toggleOption(idx)}
            className={`w-full text-left p-4 rounded-xl border-2 transition-all ${
              selected.includes(idx)
                ? 'border-primary bg-primary/5 shadow-sm'
                : 'border-border hover:border-primary/30 hover:bg-accent/50'
            }`}
          >
            <div className="flex items-center gap-3">
              <div
                className={`w-6 h-6 rounded-full border-2 flex items-center justify-center transition-all ${
                  selected.includes(idx)
                    ? 'border-primary bg-primary'
                    : 'border-muted-foreground'
```

```
              }`}
              style={selected.includes(idx) ? { borderColor: opt.color, backgroundColor: opt.color } : {}}
            >
              {selected.includes(idx) && (
                <CheckCircle2 className="w-4 h-4 text-white" />
              )}
            </div>
            <span className="text-foreground font-medium">{opt.label}</span>
          </div>
        </button>
      ))}
    </div>

    <Button onClick={handleSubmit} disabled={selected.length === 0 || submitting} className="w-full h-12 text-base
gap-2">
      <Vote className="w-4 h-4" />
      {submitting ? t('poll.submitting') : t('poll.submitVote')}
    </Button>
  </div>
 </div>
 );
}
```

文件: src/pages/QuizSubmitPage.tsx

```
import { useState, useEffect, useMemo } from 'react';
import { useParams } from 'react-router-dom';
import { useLanguage } from '@/contexts/LanguageContext';
import { supabase } from '@/integrations/supabase/client';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Textarea } from '@/components/ui/textarea';
import { CheckCircle2, ChevronLeft, ChevronRight, Send } from 'lucide-react';

interface QuizQuestion {
  type: 'single' | 'multi' | 'tf' | 'short';
  content: string;
  options: string[];
  correct_answer: string | string[];
}

interface Session {
  id: string;
  title: string;
  questions: QuizQuestion[];
  status: string;
  student_names: string[];
}

const NAME_KEY = 'quiz-student-name';
const RECENT_KEY = 'quiz-recent-names';

export default function QuizSubmitPage() {
```

```
const { sessionId } = useParams<{ sessionId: string }>();
const { t } = useLanguage();
const [session, setSession] = useState<Session | null>(null);
const [loading, setLoading] = useState(true);
const [name, setName] = useState(localStorage.getItem(NAME_KEY) || '');
const [entered, setEntered] = useState(false);
const [currentQ, setCurrentQ] = useState(0);
const [answers, setAnswers] = useState<Record<number, any>>({});
const [submitted, setSubmitted] = useState(false);
const [submitting, setSubmitting] = useState(false);
const [nameSuggestions, setNameSuggestions] = useState<string[]>([]);
const [showSuggestions, setShowSuggestions] = useState(false);

useEffect(() => {
  if (!sessionId) return;
  supabase.from('quiz_sessions').select('*').eq('id', sessionId).single()
    .then(({ data }) => {
      if (data) setSession(data as any);
      setLoading(false);
    });
}, [sessionId]);

// Build name suggestions from student_names + recent names
useEffect(() => {
  if (!session) return;
  const recent: string[] = JSON.parse(localStorage.getItem(RECENT_KEY) || '[]');
  const fromSession = Array.isArray(session.student_names) ? session.student_names : [];
  const all = [...new Set([...fromSession, ...recent])];
  setNameSuggestions(all);
}, [session]);

const filteredSuggestions = useMemo(() => {
  if (!name.trim()) return nameSuggestions.slice(0, 8);
  return nameSuggestions.filter(n => n.includes(name.trim())).slice(0, 8);
}, [name, nameSuggestions]);

const enterQuiz = () => {
  if (!name.trim()) return;
  localStorage.setItem(NAME_KEY, name.trim());
  const recent: string[] = JSON.parse(localStorage.getItem(RECENT_KEY) || '[]');
  if (!recent.includes(name.trim())) {
    recent.unshift(name.trim());
    localStorage.setItem(RECENT_KEY, JSON.stringify(recent.slice(0, 20)));
  }
  setEntered(true);
};

const setAnswer = (qi: number, value: any) => {
  setAnswers(prev => ({ ...prev, [qi]: value }));
};

const toggleMultiAnswer = (qi: number, letter: string) => {
  const current = Array.isArray(answers[qi]) ? [...answers[qi]] : [];
  if (current.includes(letter)) setAnswer(qi, current.filter((x: string) => x !== letter));
```

```
    else setAnswer(qi, [...current, letter].sort());
  };

  const submitAll = async () => {
    if (!session || submitting) return;
    setSubmitting(true);
    const questions = session.questions;
    const rows = questions.map((q, i) => {
      const ans = answers[i] ?? '';
      let isCorrect: boolean | null = null;
      if (q.type === 'single' || q.type === 'tf') {
        isCorrect = ans === q.correct_answer;
      } else if (q.type === 'multi') {
        const ca = Array.isArray(q.correct_answer) ? [...q.correct_answer].sort() : [];
        const ua = Array.isArray(ans) ? [...ans].sort() : [];
        isCorrect = JSON.stringify(ca) === JSON.stringify(ua);
      }
      // short answer: is_correct stays null for teacher review
      return {
        session_id: session.id,
        student_name: name.trim(),
        question_index: i,
        answer: ans,
        is_correct: isCorrect,
      };
    });
    await supabase.from('quiz_answers').insert(rows as any);
    setSubmitted(true);
    setSubmitting(false);
  };

  if (loading) return <div className="min-h-screen flex items-center justify-center
text-muted-foreground">{t('common.loading')}</div>;
  if (!session) return <div className="min-h-screen flex items-center justify-center
text-muted-foreground">{t('quiz.sessionNotFound')}</div>;
  if (session.status !== 'active') return (
    <div className="min-h-screen flex items-center justify-center text-muted-foreground">
      <div className="text-center">
        <div className="text-4xl mb-3"></div>
        <p className="text-lg font-medium text-foreground mb-1">{session.title}</p>
        <p className="text-sm">{t('quiz.sessionEnded')}</p>
      </div>
    </div>
  );

  if (submitted) return (
    <div className="min-h-screen flex items-center justify-center bg-background">
      <div className="text-center p-8">
        <div className="w-16 h-16 mx-auto mb-4 rounded-full bg-green-100 flex items-center justify-center">
          <CheckCircle2 className="w-8 h-8 text-green-600" />
        </div>
        <p className="text-lg font-semibold text-foreground mb-1">{t('quiz.submitSuccess')}</p>
        <p className="text-sm text-muted-foreground">{t('quiz.waitForResult')}</p>
      </div>
```

```
  </div>
  );

  // Name entry
  if (!entered) return (
    <div className="min-h-screen flex items-center justify-center bg-background p-4">
      <div className="w-full max-w-sm space-y-4">
        <div className="text-center mb-6">
          <h1 className="text-xl font-bold text-foreground mb-1">{session.title}</h1>
          <p className="text-sm text-muted-foreground">{session.questions.length} {t('quiz.questionsCount')}</p>
        </div>
        <div className="relative">
          <Input
            value={name}
            onChange={e => { setName(e.target.value); setShowSuggestions(true); }}
            onFocus={() => setShowSuggestions(true)}
            placeholder={t('quiz.enterName')}
            className="h-12 text-base"
            onKeyDown={e => e.key === 'Enter' && enterQuiz()}
          />
          {showSuggestions && filteredSuggestions.length > 0 && (
            <div className="absolute top-full left-0 right-0 z-10 mt-1 bg-card border border-border rounded-lg shadow-lg
max-h-40 overflow-auto">
              {filteredSuggestions.map(s => (
                <button key={s} className="w-full text-left px-3 py-2 text-sm hover:bg-muted transition-colors"
                  onClick={() => { setName(s); setShowSuggestions(false); }}>
                  {s}
                </button>
              ))}
            </div>
          )}
        </div>
        <Button onClick={enterQuiz} disabled={!name.trim()} className="w-full h-12 text-base">
          {t('quiz.startAnswer')}
        </Button>
      </div>
    </div>
  );

  // Quiz questions
  const questions = session.questions;
  const q = questions[currentQ];
  const answer = answers[currentQ];
  const allAnswered = questions.every((_, i) => answers[i] !== undefined && answers[i] !== '');

  return (
    <div className="min-h-screen bg-background flex flex-col">
      {/* Progress */}
      <div className="bg-card border-b border-border px-4 py-3">
        <div className="flex items-center justify-between mb-2">
          <span className="text-sm font-medium text-foreground">{session.title}</span>
          <span className="text-xs text-muted-foreground">{name}</span>
        </div>
        <div className="flex gap-1">
```

```
    {questions.map((_, i) => (
      <div key={i}
        className={`h-1.5 flex-1 rounded-full transition-colors cursor-pointer ${
          i === currentQ ? 'bg-primary' : answers[i] !== undefined ? 'bg-green-400' : 'bg-muted'
        }`}
        onClick={() => setCurrentQ(i)}
      />
    ))}
  </div>
</div>

{/* Question */}
<div className="flex-1 p-4 sm:p-6 max-w-2xl mx-auto w-full">
  <div className="mb-6">
    <span className="text-xs font-medium text-primary">Q{currentQ + 1}/{questions.length}</span>
    <p className="text-base sm:text-lg font-medium text-foreground mt-1">{q.content}</p>
  </div>

  {/* Single choice */}
  {q.type === 'single' && (
    <div className="space-y-2">
      {q.options.map((opt, i) => {
        const letter = String.fromCharCode(65 + i);
        return (
          <button key={i}
            className={`w-full text-left p-3 sm:p-4 rounded-xl border-2 transition-all ${
              answer === letter ? 'border-primary bg-primary/5' : 'border-border hover:border-muted-foreground/30'
            }`}
            onClick={() => setAnswer(currentQ, letter)}
          >
            <span className="font-mono text-sm mr-2 text-muted-foreground">{letter}.</span>
            <span className="text-sm text-foreground">{opt}</span>
          </button>
        );
      })}
    </div>
  )}

  {/* Multiple choice */}
  {q.type === 'multi' && (
    <div className="space-y-2">
      {q.options.map((opt, i) => {
        const letter = String.fromCharCode(65 + i);
        const selected = Array.isArray(answer) && answer.includes(letter);
        return (
          <button key={i}
            className={`w-full text-left p-3 sm:p-4 rounded-xl border-2 transition-all ${
              selected ? 'border-primary bg-primary/5' : 'border-border hover:border-muted-foreground/30'
            }`}
            onClick={() => toggleMultiAnswer(currentQ, letter)}
          >
            <span className="font-mono text-sm mr-2 text-muted-foreground">{letter}.</span>
            <span className="text-sm text-foreground">{opt}</span>
          </button>
```

```
          );
        })}
        <p className="text-xs text-muted-foreground">{t('quiz.multiHint')}</p>
      </div>
    )}

    {/* True/False */}
    {q.type === 'tf' && (
      <div className="grid grid-cols-2 gap-3">
        <button
          className={`p-6 rounded-xl border-2 text-center transition-all ${
            answer === 'A' ? 'border-green-500 bg-green-50' : 'border-border hover:border-muted-foreground/30'
          }`}
          onClick={() => setAnswer(currentQ, 'A')}
        >
          <span className="text-2xl"></span>
          <p className="text-sm mt-1 text-foreground">{t('quiz.true')}</p>
        </button>
        <button
          className={`p-6 rounded-xl border-2 text-center transition-all ${
            answer === 'B' ? 'border-red-500 bg-red-50' : 'border-border hover:border-muted-foreground/30'
          }`}
          onClick={() => setAnswer(currentQ, 'B')}
        >
          <span className="text-2xl"></span>
          <p className="text-sm mt-1 text-foreground">{t('quiz.false')}</p>
        </button>
      </div>
    )}

    {/* Short answer */}
    {q.type === 'short' && (
      <Textarea
        value={answer || ''}
        onChange={e => setAnswer(currentQ, e.target.value)}
        placeholder={t('quiz.shortPlaceholder')}
        rows={4}
        className="text-base"
      />
    )}
  </div>

  {/* Navigation */}
  <div className="bg-card border-t border-border px-4 py-3 flex items-center justify-between">
    <Button variant="outline" size="sm" onClick={() => setCurrentQ(Math.max(0, currentQ - 1))} disabled={currentQ
=== 0}>
      <ChevronLeft className="w-4 h-4 mr-1" /> {t('quiz.prev')}
    </Button>
    {currentQ < questions.length - 1 ? (
      <Button size="sm" onClick={() => setCurrentQ(currentQ + 1)}>
        {t('quiz.next')} <ChevronRight className="w-4 h-4 ml-1" />
      </Button>
    ) : (
      <Button size="sm" onClick={submitAll} disabled={!allAnswered || submitting} className="gap-1">
```

```
          <Send className="w-4 h-4" /> {t('quiz.submit')}
        </Button>
      )}
    </div>
  </div>
 );
}
```

文件: src/pages/ResetPassword.tsx

```
import { useState, useEffect } from 'react';
import { useLanguage } from '@/contexts/LanguageContext';
import { supabase } from '@/integrations/supabase/client';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { toast } from '@/hooks/use-toast';
import { Lock } from 'lucide-react';
import { useNavigate } from 'react-router-dom';

export default function ResetPassword() {
  const { t } = useLanguage();
  const [password, setPassword] = useState('');
  const [loading, setLoading] = useState(false);
  const [ready, setReady] = useState(false);
  const navigate = useNavigate();

  useEffect(() => {
    const hash = window.location.hash;
    if (hash.includes('type=recovery')) {
      setReady(true);
    }
  }, []);

  const handleReset = async () => {
    if (!password || password.length < 6) {
      toast({ title: t('reset.minLength'), variant: 'destructive' });
      return;
    }
    setLoading(true);
    const { error } = await supabase.auth.updateUser({ password });
    setLoading(false);
    if (error) {
      toast({ title: t('reset.failed'), description: error.message, variant: 'destructive' });
    } else {
      toast({ title: t('reset.success') });
      navigate('/');
    }
  };

  if (!ready) {
    return (
      <div className="min-h-screen bg-surface flex items-center justify-center p-4">
        <div className="text-center space-y-4">
```

```
        <p className="text-muted-foreground">{t('reset.invalidLink')}</p>
        <Button variant="outline" onClick={() => navigate('/auth')}>{t('reset.backToLogin')}</Button>
      </div>
    </div>
  );
}

return (
  <div className="min-h-screen bg-surface flex items-center justify-center p-4">
    <div className="w-full max-w-sm space-y-6">
      <div className="text-center">
        <h1 className="text-xl font-bold text-foreground">{t('reset.title')}</h1>
      </div>
      <div className="bg-card border border-border rounded-xl p-6 space-y-4">
        <div className="relative">
          <Lock className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-muted-foreground" />
          <Input
            type="password"
            placeholder={t('reset.placeholder')}
            value={password}
            onChange={e => setPassword(e.target.value)}
            onKeyDown={e => e.key === 'Enter' && handleReset()}
            className="pl-10"
          />
        </div>
        <Button onClick={handleReset} disabled={loading} className="w-full">
          {loading ? t('reset.resetting') : t('reset.confirm')}
        </Button>
      </div>
    </div>
  </div>
);
}
```

文件: src/pages/SeatCheckinPage.tsx

```
import { useState, useEffect, useMemo } from 'react';
import { useParams } from 'react-router-dom';
import { useLanguage } from '@/contexts/LanguageContext';
import { supabase } from '@/integrations/supabase/client';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { toast } from '@/hooks/use-toast';
import { MapPin, CheckCircle2 } from 'lucide-react';
import ClassroomCheckinView from '@/components/checkin-views/ClassroomCheckinView';
import RoundTableCheckinView from '@/components/checkin-views/RoundTableCheckinView';
import ConferenceCheckinView from '@/components/checkin-views/ConferenceCheckinView';
import ConcertCheckinView from '@/components/checkin-views/ConcertCheckinView';
import ComputerLabCheckinView from '@/components/checkin-views/ComputerLabCheckinView';

export default function SeatCheckinPage() {
  const { sessionId } = useParams<{ sessionId: string }>();
  const { t } = useLanguage();
```

```
const [session, setSession] = useState<{
  seat_data: unknown;
  student_names: string[];
  scene_config: Record<string, unknown>;
  scene_type: string;
} | null>(null);
const [loading, setLoading] = useState(true);
const [name, setName] = useState('');
const [suggestions, setSuggestions] = useState<string[]>([]);
const [checkedIn, setCheckedIn] = useState(false);
const [submitting, setSubmitting] = useState(false);

useEffect(() => {
  if (!sessionId) return;
  supabase.from('seat_checkin_sessions').select('*').eq('id', sessionId).single()
    .then(({ data, error }) => {
      if (error || !data) {
        toast({ title: t('seatCheckin.sessionNotFound'), variant: 'destructive' });
        setLoading(false);
        return;
      }
      setSession({
        seat_data: data.seat_data,
        student_names: data.student_names as unknown as string[],
        scene_config: data.scene_config as unknown as Record<string, unknown>,
        scene_type: (data as Record<string, unknown>).scene_type as string || 'classroom',
      });
      setLoading(false);
    });
}, [sessionId]);

const handleNameInput = (val: string) => {
  setName(val);
  if (!session || val.length === 0) { setSuggestions([]); return; }
  const filtered = session.student_names.filter(n =>
    n.toLowerCase().includes(val.toLowerCase()) && n !== val
  );
  setSuggestions(filtered.slice(0, 5));
};

const handleSubmit = async () => {
  if (!name.trim() || !sessionId || !session) return;
  if (!session.student_names.includes(name.trim())) {
    toast({ title: t('seatCheckin.nameNotFound'), description: t('seatCheckin.checkName'), variant: 'destructive' });
    return;
  }
  setSubmitting(true);
  try {
    await supabase.from('seat_checkin_records').insert({
      session_id: sessionId,
      student_name: name.trim(),
    });
    setCheckedIn(true);
  } catch {
```

```
    toast({ title: t('seatCheckin.failed'), variant: 'destructive' });
  } finally {
    setSubmitting(false);
  }
};

if (loading) {
  return <div className="flex items-center justify-center min-h-screen
text-muted-foreground">{t('seatCheckin.loading')}</div>;
}

if (!session) {
  return <div className="flex items-center justify-center min-h-screen
text-muted-foreground">{t('seatCheckin.notFound')}</div>;
}

if (!checkedIn) {
  return (
    <div className="min-h-screen flex items-center justify-center bg-background p-4">
      <div className="w-full max-w-sm space-y-6">
        <div className="text-center space-y-2">
          <MapPin className="w-12 h-12 mx-auto text-primary" />
          <h1 className="text-xl font-bold text-foreground">{t('seatCheckin.title')}</h1>
          <p className="text-sm text-muted-foreground">{t('seatCheckin.desc')}</p>
        </div>
        <div className="relative">
          <Input
            value={name}
            onChange={e => handleNameInput(e.target.value)}
            placeholder={t('seatCheckin.namePlaceholder')}
            className="text-center text-lg h-12"
            onKeyDown={e => e.key === 'Enter' && handleSubmit()}
          />
          {suggestions.length > 0 && (
            <div className="absolute z-10 w-full mt-1 bg-card border border-border rounded-lg shadow-lg
overflow-hidden">
              {suggestions.map(s => (
                <button key={s} onClick={() => { setName(s); setSuggestions([]); }} className="w-full text-left px-4 py-2.5
text-sm hover:bg-muted transition-colors">
                  {s}
                </button>
              ))}
            </div>
          )}
        </div>
        <Button onClick={handleSubmit} disabled={!name.trim() || submitting} className="w-full h-12 text-base">
          {submitting ? t('seatCheckin.checking') : t('seatCheckin.confirm')}
        </Button>
      </div>
    </div>
  );
}

const sceneType = session.scene_type;
```

```
  const studentName = name.trim();

  return (
    <div className="min-h-screen bg-background p-4 overflow-auto">
      <div className="max-w-2xl mx-auto space-y-4">
        <div className="text-center space-y-1">
          <div className="flex items-center justify-center gap-2 text-primary">
            <CheckCircle2 className="w-6 h-6" />
            <span className="text-lg font-bold">{t('seatCheckin.success')}</span>
          </div>
        </div>

        {sceneType === 'classroom' && (
          <ClassroomCheckinView seatData={session.seat_data} sceneConfig={session.scene_config}
studentName={studentName} />
        )}
        {(sceneType === 'smartClassroom' || sceneType === 'banquet') && (
          <RoundTableCheckinView seatData={session.seat_data} sceneConfig={session.scene_config}
studentName={studentName} sceneType={sceneType} />
        )}
        {sceneType === 'conference' && (
          <ConferenceCheckinView seatData={session.seat_data} sceneConfig={session.scene_config}
studentName={studentName} />
        )}
        {sceneType === 'concertHall' && (
          <ConcertCheckinView seatData={session.seat_data} sceneConfig={session.scene_config}
studentName={studentName} />
        )}
        {sceneType === 'computerLab' && (
          <ComputerLabCheckinView seatData={session.seat_data} sceneConfig={session.scene_config}
studentName={studentName} />
        )}
      </div>
    </div>
  );
}
```

文件: src/test/example.test.ts

```
----------------------------------------
import { describe, it, expect } from "vitest";

describe("example", () => {
  it("should pass", () => {
    expect(true).toBe(true);
  });
});
```

文件: src/test/setup.ts

```
----------------------------------------
import "@testing-library/jest-dom";

Object.defineProperty(window, "matchMedia", {
```

```
  writable: true,
  value: (query: string) => ({
    matches: false,
    media: query,
    onchange: null,
    addListener: () => {},
    removeListener: () => {},
    addEventListener: () => {},
    removeEventListener: () => {},
    dispatchEvent: () => {},
  }),
});

// Polyfill ResizeObserver for Radix UI components
class ResizeObserverMock {
  observe() {}
  unobserve() {}
  disconnect() {}
}
window.ResizeObserver = ResizeObserverMock as any;

// Polyfill SpeechSynthesisUtterance
if (typeof window.SpeechSynthesisUtterance === 'undefined') {
  (window as any).SpeechSynthesisUtterance = class {
    text = '';
    lang = '';
    rate = 1;
    volume = 1;
    constructor(text?: string) { this.text = text || ''; }
  };
}
if (typeof window.speechSynthesis === 'undefined') {
  (window as any).speechSynthesis = { speak: () => {}, cancel: () => {}, getVoices: () => [] };
}
```

文件: src/vite-env.d.ts

```
/// <reference types="vite/client" />
```

文件: src/App.tsx

```
import { Toaster } from "@/components/ui/toaster";
import { Toaster as Sonner } from "@/components/ui/sonner";
import { TooltipProvider } from "@/components/ui/tooltip";
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import { BrowserRouter, Routes, Route } from "react-router-dom";
import { lazy, Suspense } from "react";
import { AuthProvider } from "@/contexts/AuthContext";
import { LanguageProvider } from "@/contexts/LanguageContext";
import { FeatureConfigProvider } from "@/contexts/FeatureConfigContext";
import Index from "./pages/Index";
import NotFound from "./pages/NotFound";
```

```
const DiscussPage = lazy(() => import("./pages/DiscussPage"));
const CheckInPage = lazy(() => import("./pages/CheckInPage"));
const AuthPage = lazy(() => import("./pages/AuthPage"));
const ResetPassword = lazy(() => import("./pages/ResetPassword"));
const AdminPage = lazy(() => import("./pages/AdminPage"));
const GoRedirect = lazy(() => import("./pages/GoRedirect"));
const SeatCheckinPage = lazy(() => import("./pages/SeatCheckinPage"));
const BoardPage = lazy(() => import("./pages/BoardPage"));
const BoardSubmitPage = lazy(() => import("./pages/BoardSubmitPage"));
const QuizSubmitPage = lazy(() => import("./pages/QuizSubmitPage"));
const PollVotePage = lazy(() => import("./pages/PollVotePage"));

const queryClient = new QueryClient();

const Loading = () => <div className="flex items-center justify-center min-h-screen
text-muted-foreground">Loading...</div>;

const App = () => (
  <LanguageProvider>
    <QueryClientProvider client={queryClient}>
      <AuthProvider>
        <FeatureConfigProvider>
          <TooltipProvider>
            <Toaster />
            <Sonner />
            <BrowserRouter>
              <Routes>
                <Route path="/" element={<Index />} />
                <Route path="/auth" element={<Suspense fallback={<Loading />}><AuthPage /></Suspense>} />
                <Route path="/reset-password" element={<Suspense fallback={<Loading />}><ResetPassword
/></Suspense>} />
                <Route path="/admin" element={<Suspense fallback={<Loading />}><AdminPage /></Suspense>} />
                <Route path="/discuss/:topicId" element={<Suspense fallback={<Loading />}><DiscussPage /></Suspense>}
/>
                <Route path="/checkin/:sessionId" element={<Suspense fallback={<Loading />}><CheckInPage
/></Suspense>} />
                <Route path="/go" element={<Suspense fallback={<Loading />}><GoRedirect /></Suspense>} />
                <Route path="/seat-checkin/:sessionId" element={<Suspense fallback={<Loading />}><SeatCheckinPage
/></Suspense>} />
                <Route path="/board/:boardId" element={<Suspense fallback={<Loading />}><BoardPage /></Suspense>} />
                <Route path="/board/:boardId/submit" element={<Suspense fallback={<Loading />}><BoardSubmitPage
/></Suspense>} />
                <Route path="/quiz/:sessionId" element={<Suspense fallback={<Loading />}><QuizSubmitPage
/></Suspense>} />
                <Route path="/poll/:pollId" element={<Suspense fallback={<Loading />}><PollVotePage /></Suspense>} />
                <Route path="*" element={<NotFound />} />
              </Routes>
            </BrowserRouter>
          </TooltipProvider>
        </FeatureConfigProvider>
      </AuthProvider>
    </QueryClientProvider>
  </LanguageProvider>
```

```
);

export default App;
```